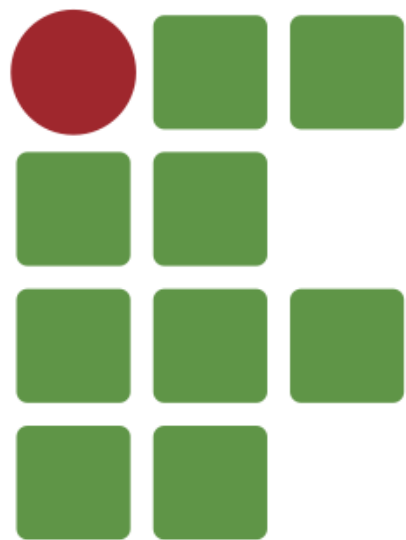




**INSTITUTO
FEDERAL**
Brasília

ARQUITETURA DE COMPUTADORES

Prof. MSc. Thiago Neves
IFB Campus Taguatinga
22/08/2026

AULA 03

Ciclo da instrução

Estruturas de Interconexão

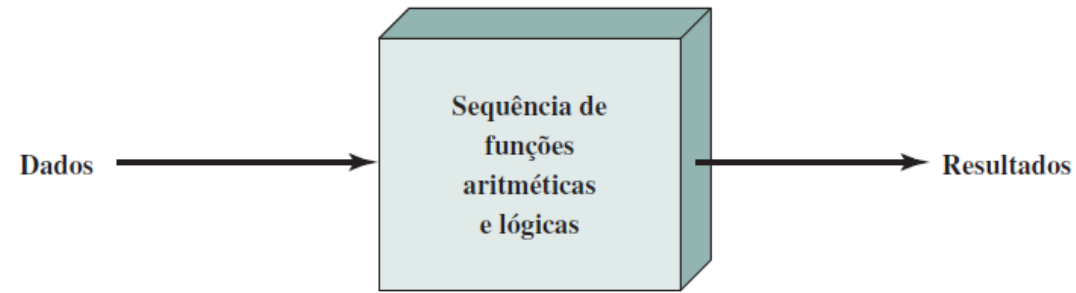
Memória Principal, Memória Cache

Compilação e Interpretação

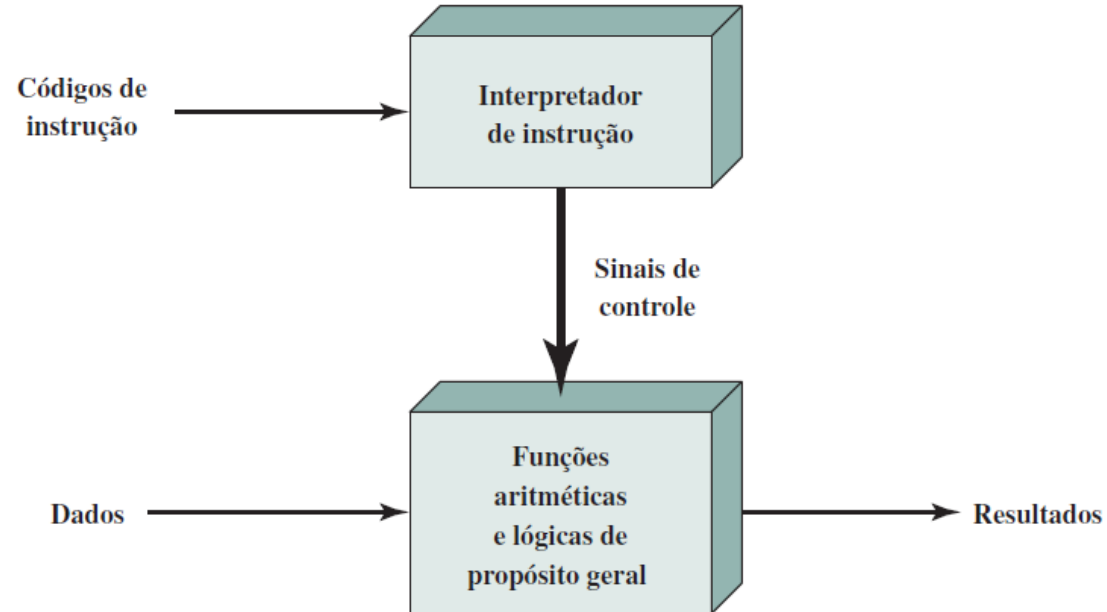
Arquitetura de von Neumann

- ❑ Dados e instruções são armazenados em uma única memória de leitura e escrita.
- ❑ O conteúdo dessa memória é endereçável pela localização, sem considerar o tipo de dados contido ali.
- ❑ A execução ocorre em um padrão sequencial (a menos que modificado explicitamente) de uma instrução para a seguinte.

Abordagens de hardware e software

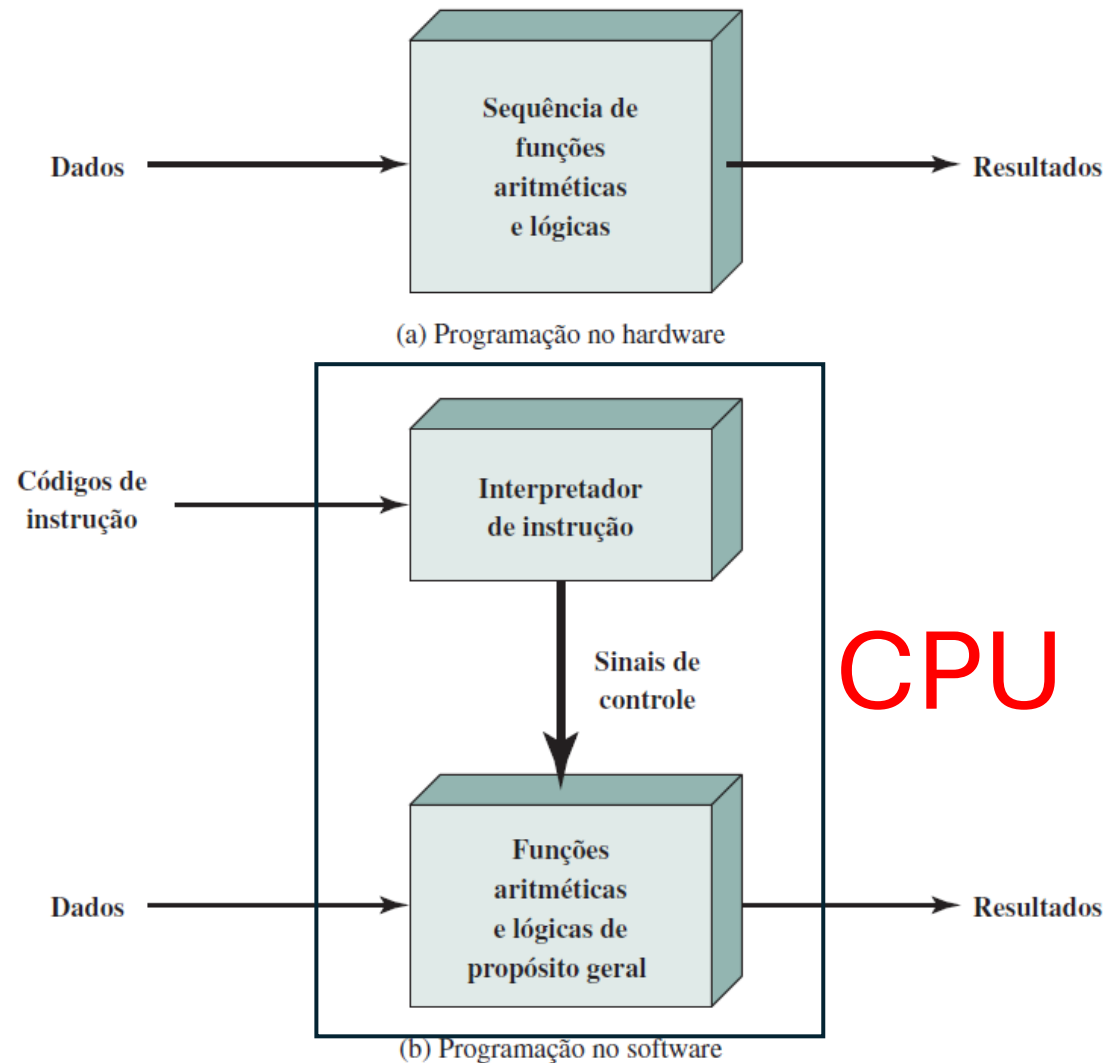


(a) Programação no hardware

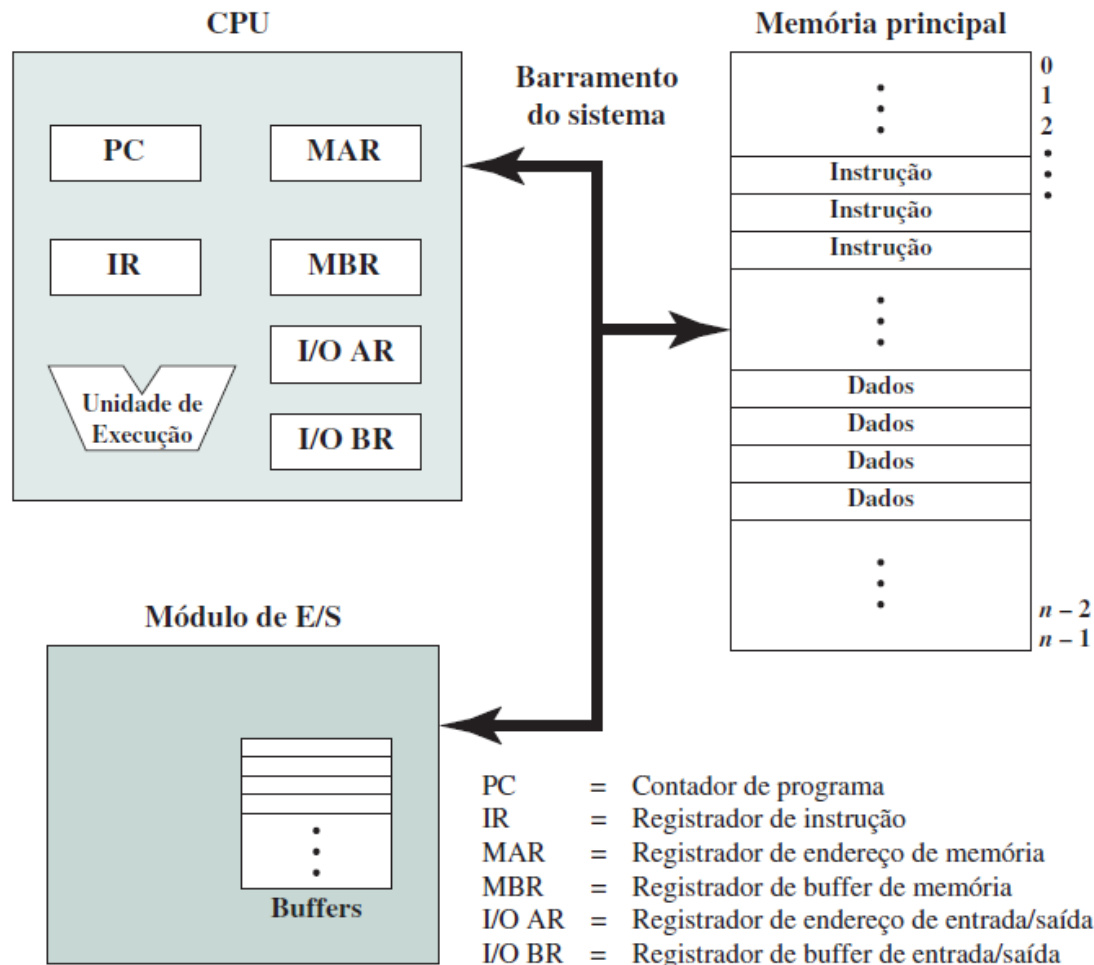


(b) Programação no software

Abordagens de hardware e software



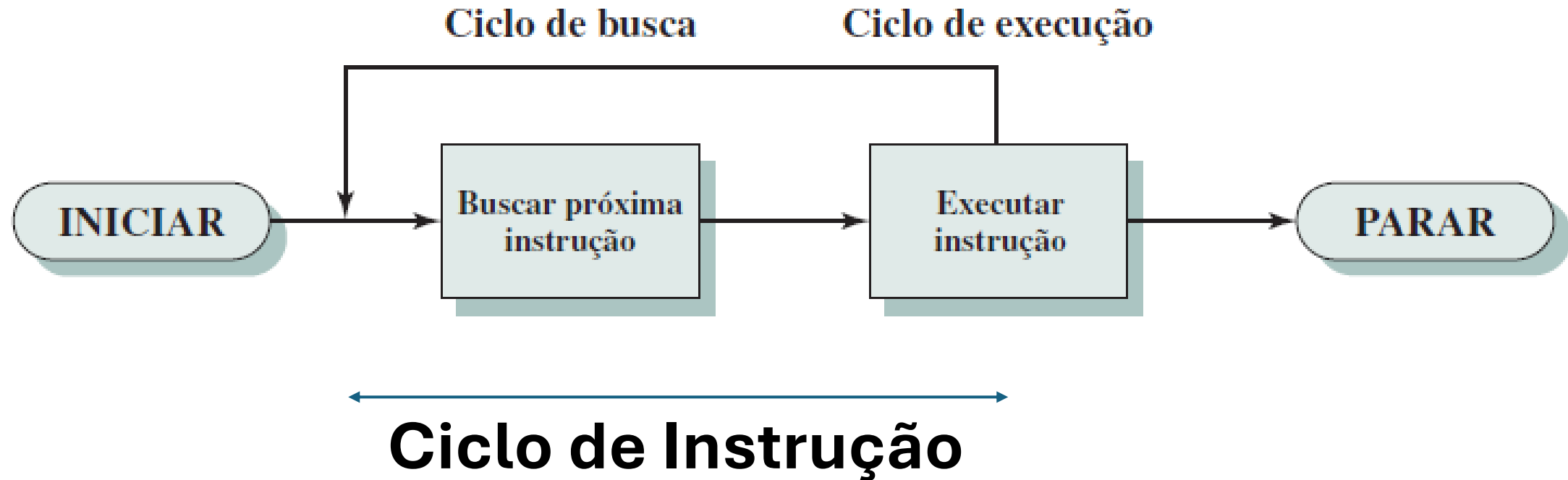
Componentes do computador



A CPU troca dados com a memória. Para essa finalidade, ela normalmente utiliza dois registradores internos (a CPU): um **registrador de endereço de memória (MAR)**, que especifica o endereço na memória para a próxima leitura ou escrita, e um **registrador de buffer de memória (MBR)**, que contém os dados a serem escritos na memória ou recebe os dados lidos da memória.

De modo semelhante, um registrador de endereço de E/S (I/O AR) especifica um dispositivo de E/S em particular. Um registrador de buffer de E/S (I/O BR) é usado para a troca de dados entre um módulo de E/S e a CPU.

Função do Computador

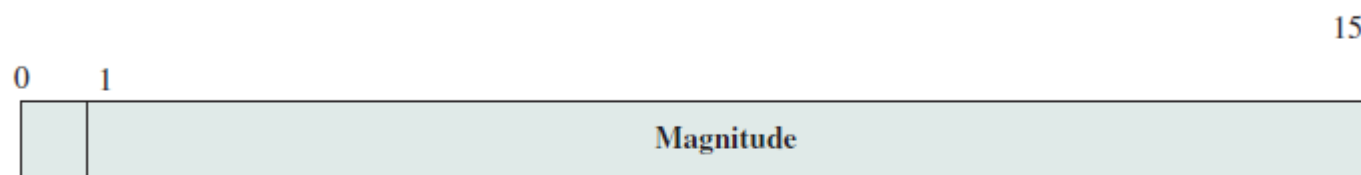


Um Computador de 16bits

Máquina Hipotética de 16 bits



(a) Formato de instrução



(b) Formato inteiro

Contador de programa (PC) = endereço de instrução
Registrador de instrução (IR) = instrução sendo executada
Acumulador (AC) = armazenamento temporário

(c) Registradores internos da CPU

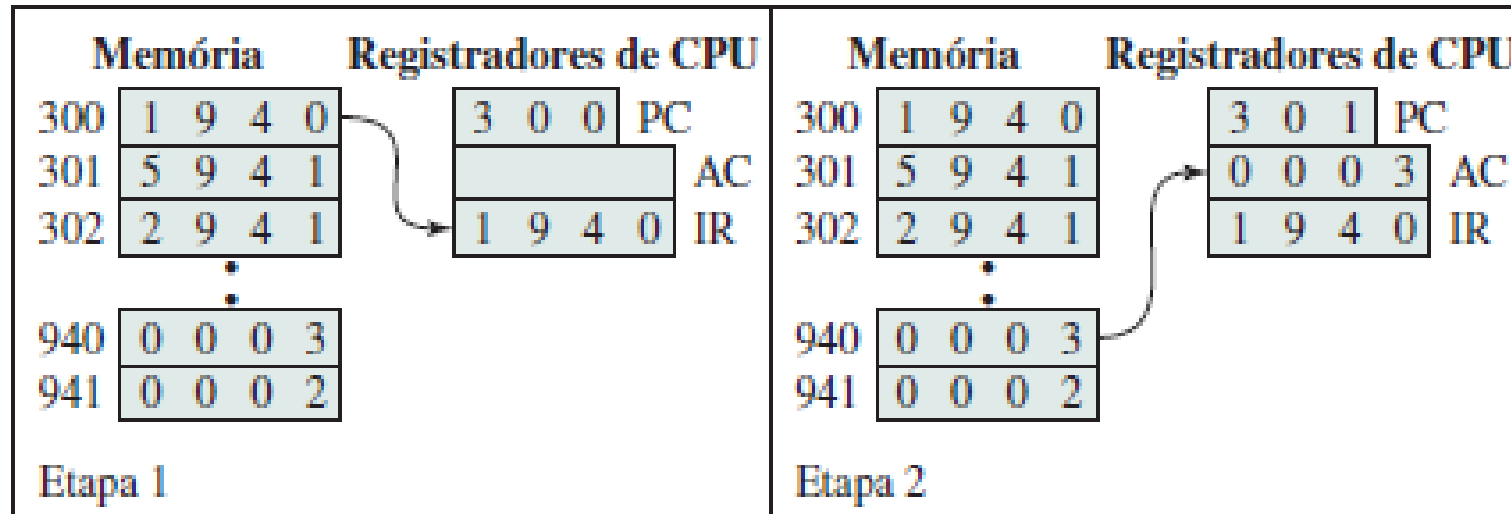
0001 = carrega AC da memória
0010 = armazena AC na memória
0101 = adiciona ao AC da memória

(d) Lista parcial de opcodes

O processador contém um único registrador de dados, chamado acumulador (AC). Instruções e dados possuem 16 bits de extensão. Assim, é conveniente organizar a memória usando palavras de 16 bits.

O formato de instrução oferece 4 bits para o opcode, de modo que pode haver até $2^4 = 16$ opções diferentes, e até $2^{12} = 4096$ (4K) palavras de memória podem ser endereçadas diretamente.

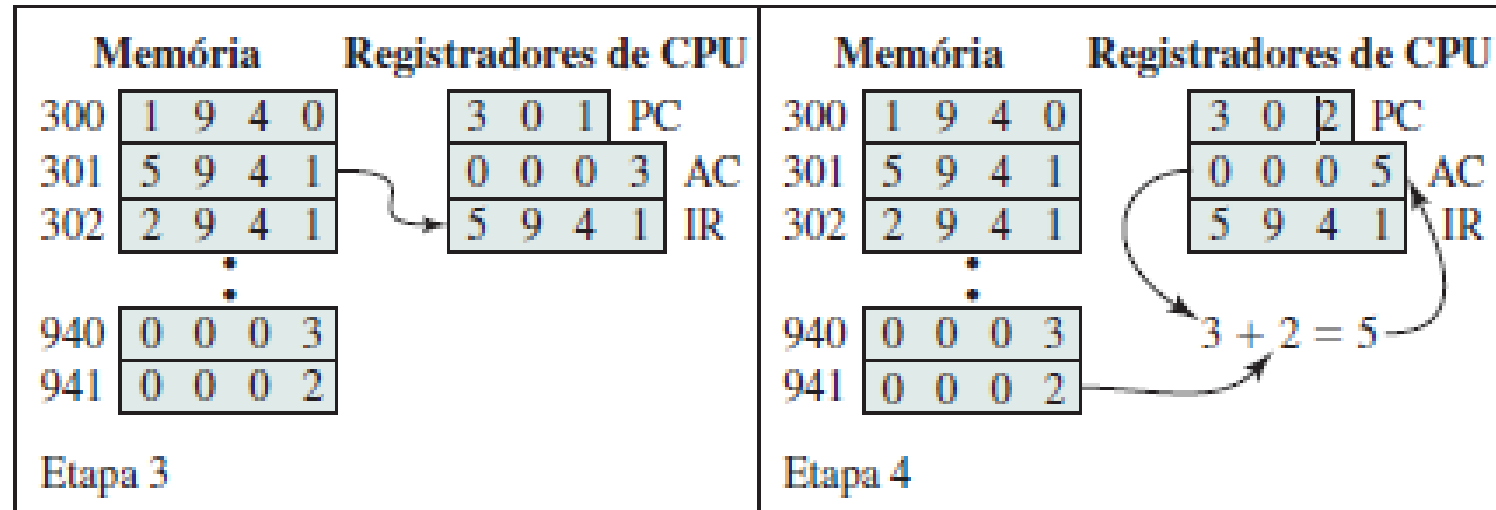
Exemplo da execução do programa – Parte 1



O PC contém 300, o endereço da primeira instrução. Essa instrução (o valor 1940 em hexadecimal) é carregada no registrador de instrução IR, e o PC é incrementado.

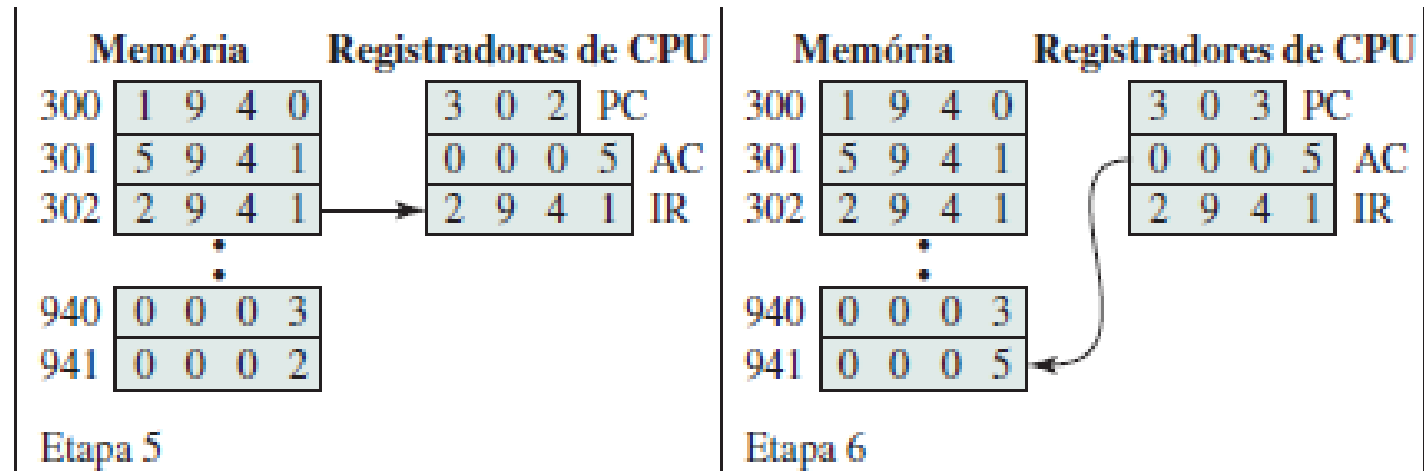
Os 4 primeiros bits (primeiro dígito hexadecimal) no IR indicam que o AC deve ser carregado. Os 12 bits restantes (três dígitos hexadecimais) especificam o endereço (940) de onde os dados devem ser carregados.

Exemplo da execução do programa – Parte 2



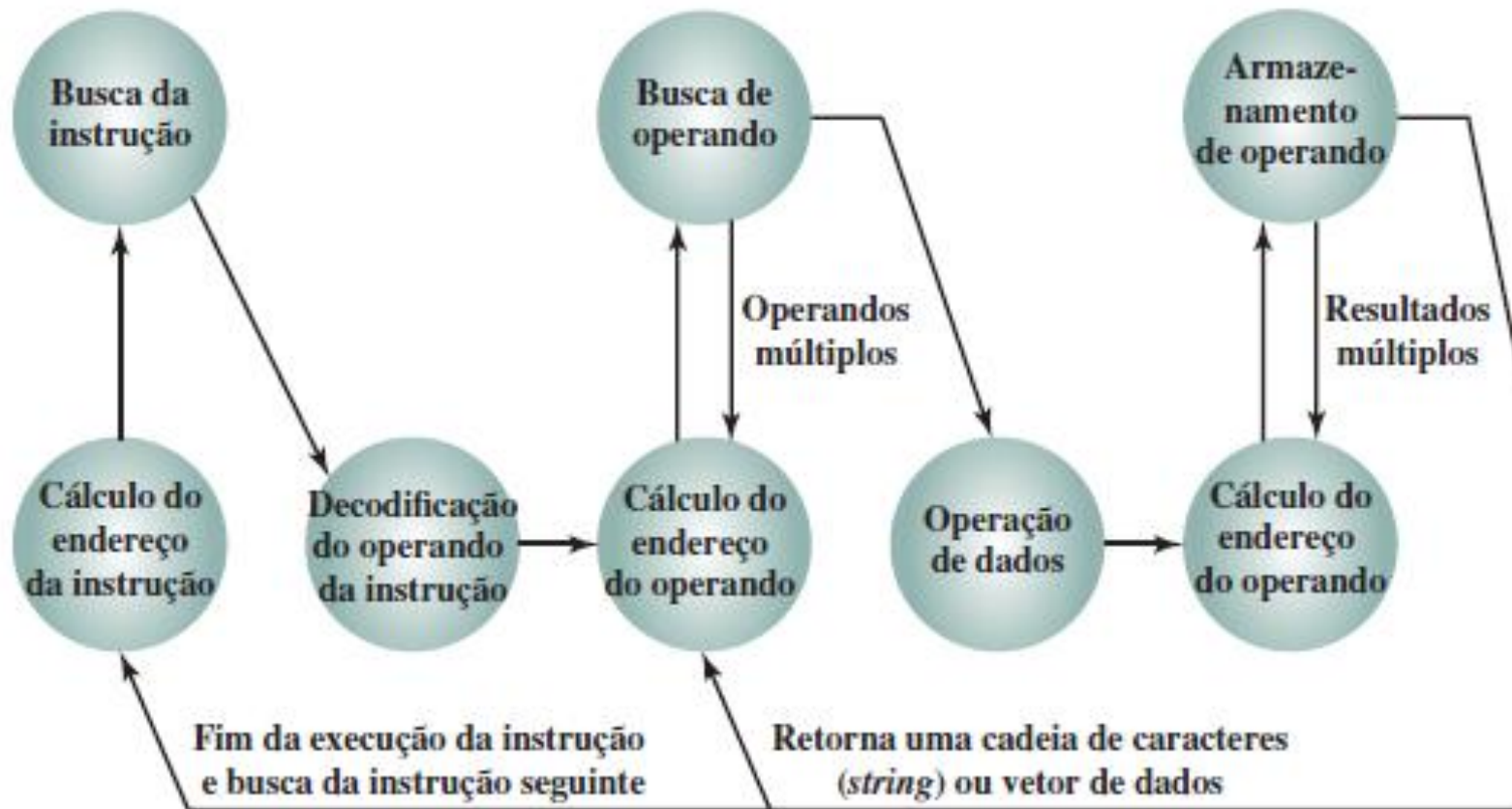
A próxima instrução (5941) é buscada do local 301, e o PC é incrementado. O conteúdo antigo do AC e o conteúdo do local 941 são somados, e o resultado é armazenado no AC.

Exemplo da execução do programa – Parte 3



A próxima instrução (2941) e busca do local 302, e o PC é incrementado. O conteúdo do AC é armazenado no local 941.

Diagrama de estado de ciclo de instrução (Processador PDP-11)



PDP-11 é uma famosa série de minicomputadores de 16 bits desenvolvida pela Digital Equipment Corporation (DEC) a partir de 1970, marcando profundamente a história da computação ao servir como plataforma pioneira para o sistema operacional Unix e influenciar arquiteturas modernas de microprocessadores.

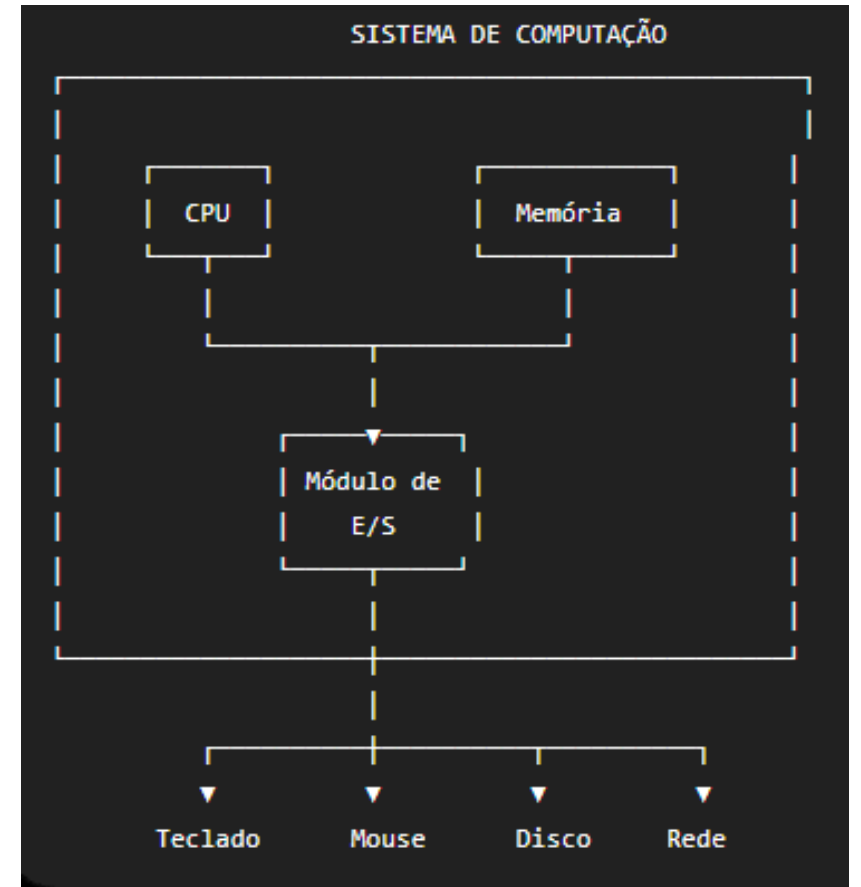
Estruturas de Interconexão

Módulos de Memória

Normalmente, um módulo de memória consiste em N palavras de mesmo tamanho. Cada palavra recebe um endereço numérico exclusivo $(0, 1, \dots, N - 1)$. Uma palavra de dados pode ser lida ou escrita na memória. A natureza da operação é indicada por sinais de controle de leitura e escrita. O local para a operação é especificado por um endereço.

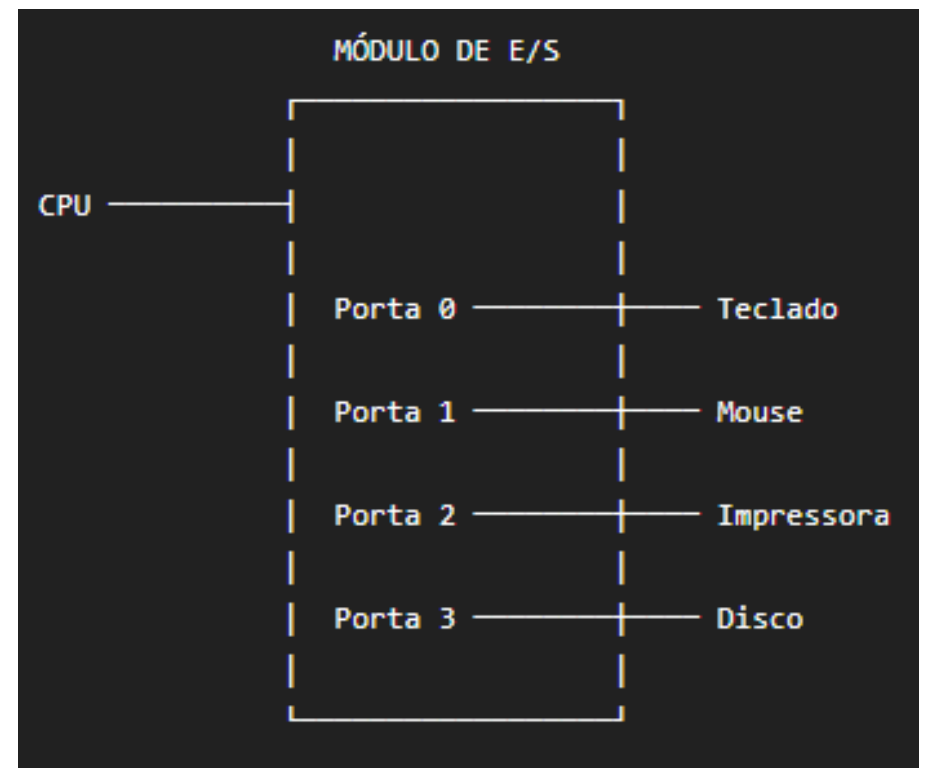
Módulo de E/S (Entrada/Saída)

Partindo de um ponto de vista interno (ao sistema de computação), **a E/S é funcionalmente semelhante à memória**. Existem duas operações, leitura e escrita. Além disso, um módulo de E/S pode controlar mais de um dispositivo externo.



Módulo de E/S (Entrada/Saída)

Podemos nos referir a cada uma das interfaces para um dispositivo externo como uma porta dando a cada uma um endereço exclusivo (por exemplo, 0, 1, ..., $M - 1$). Além disso, existem caminhos de dados externos para entrada e saída de dados com um dispositivo externo. Por fim, um módulo de E/S pode ser capaz de enviar sinais de interrupção ao processador.

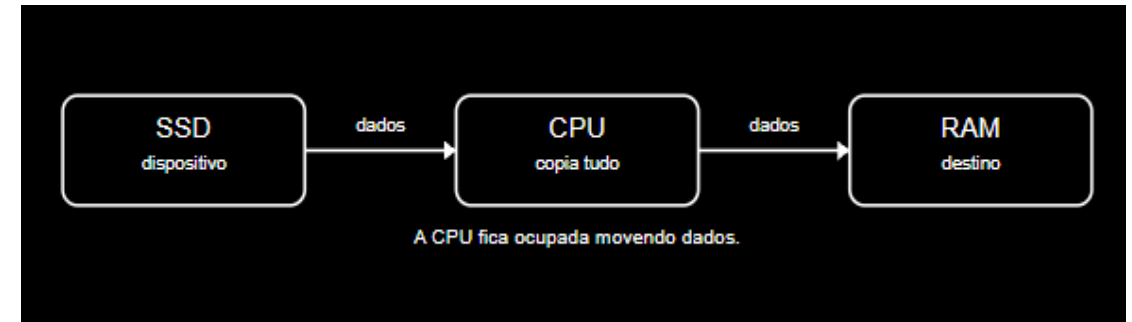


Estrutura de interconexão: tipos de transferência

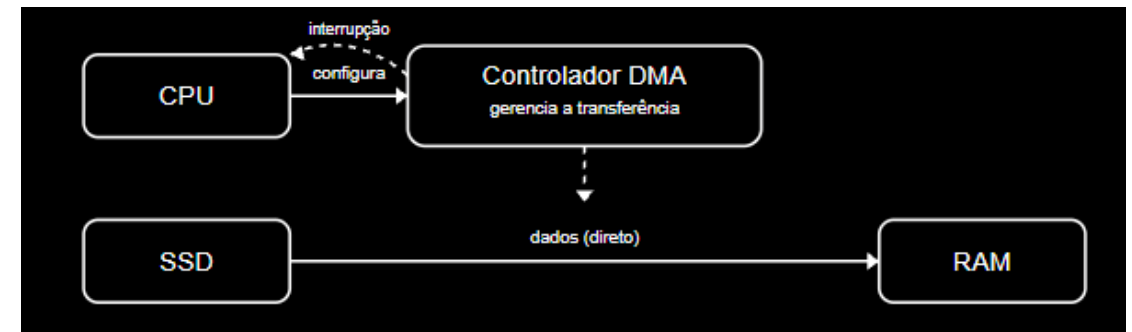
- ❑ **Memória para processador:** o processador lê uma instrução ou uma unidade de dados da memória.
- ❑ **Processador para memória:** o processador escreve uma unidade de dados na memória.
- ❑ **E/S para processador:** o processador lê dados de um dispositivo de E/S por meio de um módulo de E/S.
- ❑ **Processador para E/S:** o processador envia dados para o dispositivo de E/S.
- ❑ **E/S de/ou para a memória:** para esses dois casos, um módulo de E/S tem permissão para trocar dados diretamente com a memória, sem passar pelo processador, usando o DMA. !!!!!!!

DMA (Direct Memory Access)

É um mecanismo que permite que um dispositivo de entrada/saída transfira dados diretamente para a memória RAM, sem que a CPU precise copiar cada byte. O papel da CPU passa a ser apenas iniciar a transferência e ser avisada quando ela termina.

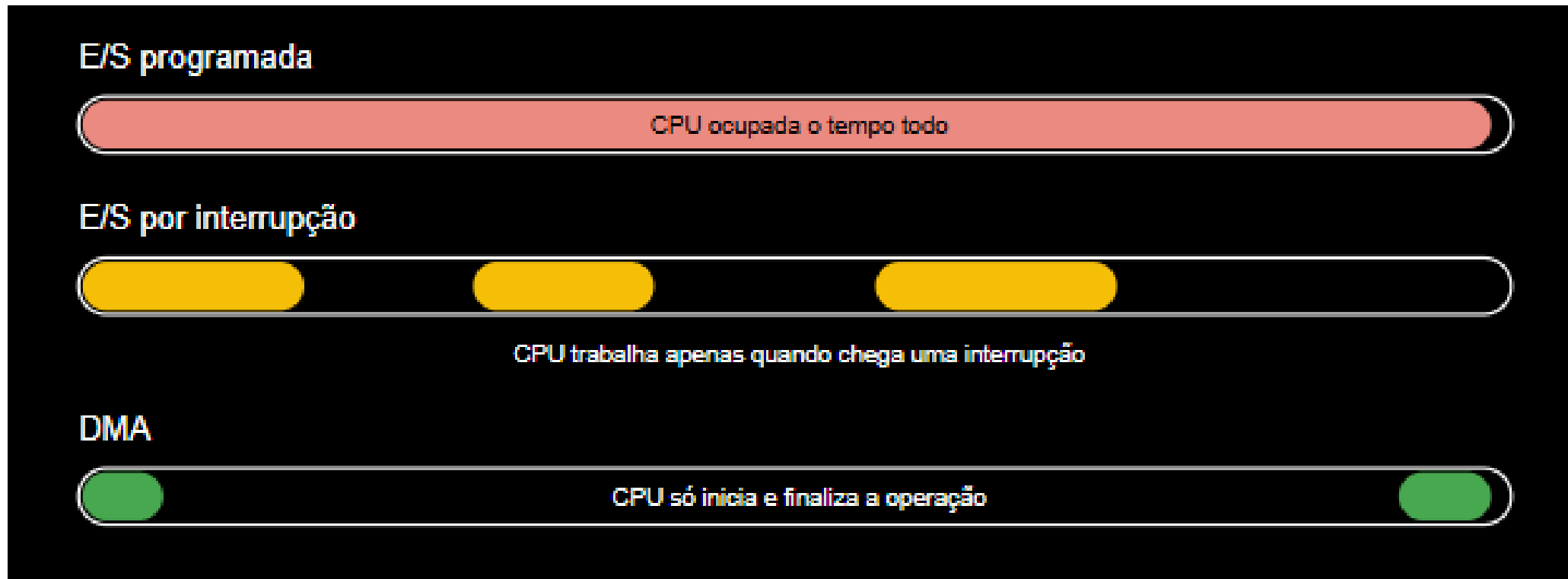


(Sem DMA)



(Com DMA)

DMA (Direct Memory Access)



Interconexão de Barramento (Bus)

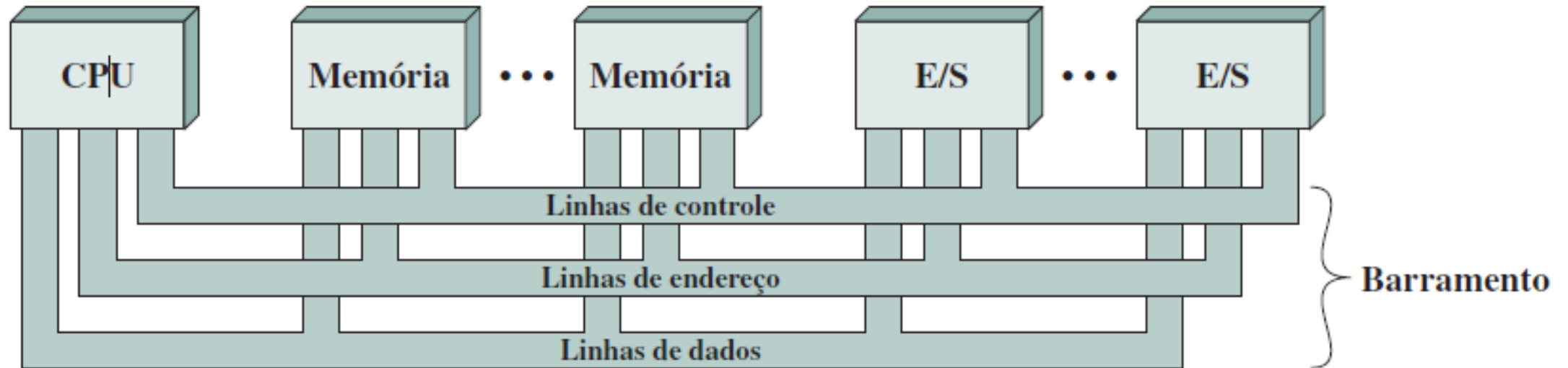
Um barramento é um caminho de comunicação que conecta dois ou mais dispositivos. Uma característica-chave de um barramento é que ele é um meio de transmissão compartilhado.

Diversos dispositivos conectam-se ao barramento, e um sinal transmitido por qualquer um dos dispositivos está disponível para recepção por todos os outros dispositivos conectados ao barramento. Se dois dispositivos transmitem durante o mesmo período de tempo, seus sinais serão sobrepostos e ficarão distorcidos. Assim, somente um dispositivo de cada vez pode transmitir com sucesso.

Interconexão de Barramento

Os sistemas de computação contêm diversos barramentos diferentes, que oferecem caminhos entre os componentes em diversos níveis da hierarquia do sistema de computação. **Um barramento que conecta os principais componentes do computador (processador, memória, E/S) é chamado de barramento do sistema.** As estruturas de interconexão de computador mais comuns são baseadas no uso de um ou mais barramentos do sistema.

Interconexão de Barramento



Interconexão de Barramento

- ❑ As **linhas de dados** oferecem um caminho para movimentação de dados entre os módulos do sistema.
- ❑ As **linhas de endereço** são usadas para designar a origem ou o destino dos dados no barramento de dados.
- ❑ As **linhas de controle** são usadas para controlar o acesso e o uso das linhas de dados e endereço. As linhas de controle típicas incluem:
 - ❑ **Escrita de memória**: permite que os dados no barramento sejam escritos no local endereçado.
 - ❑ **Leitura de memória**: permite que os dados do local endereçado sejam colocados no barramento.
 - ❑ **Escrita de E/S**: permite que os dados no barramento sejam enviados para a porta de E/S endereçada.
 - ❑ **Leitura de E/S**: permite que os dados da porta de E/S endereçada sejam colocados no barramento.
 - ❑ **ACK de transferência**: indica que dados foram aceitos ou colocados no barramento.
 - ❑ **Solicitação de barramento (*bus request*)**: indica que um módulo precisa obter controle do barramento.
 - ❑ **Concessão de barramento (*bus grant*)**: indica que um módulo solicitante recebeu controle do barramento.
 - ❑ **Requisição de interrupção (*interrupt request*)**: indica que uma interrupção está pendente.
 - ❑ **ACK de interrupção (*interrupt acknowledge*)**: confirma que a interrupção pendente foi reconhecida.
 - ❑ **Clock**: é usado para operações de sincronização.
 - ❑ **Reset**: inicializa todos os módulos.

Interconexão Ponto a Ponto

A arquitetura de barramento compartilhada foi por décadas a técnica padrão para a interconexão entre o processador e outros componentes (memória, E/S e assim por diante). Mas os sistemas contemporâneos se baseiam cada vez mais na interconexão de sistema ponto a ponto, em vez de se basearem nos barramentos compartilhados.

Interconexão Ponto a Ponto

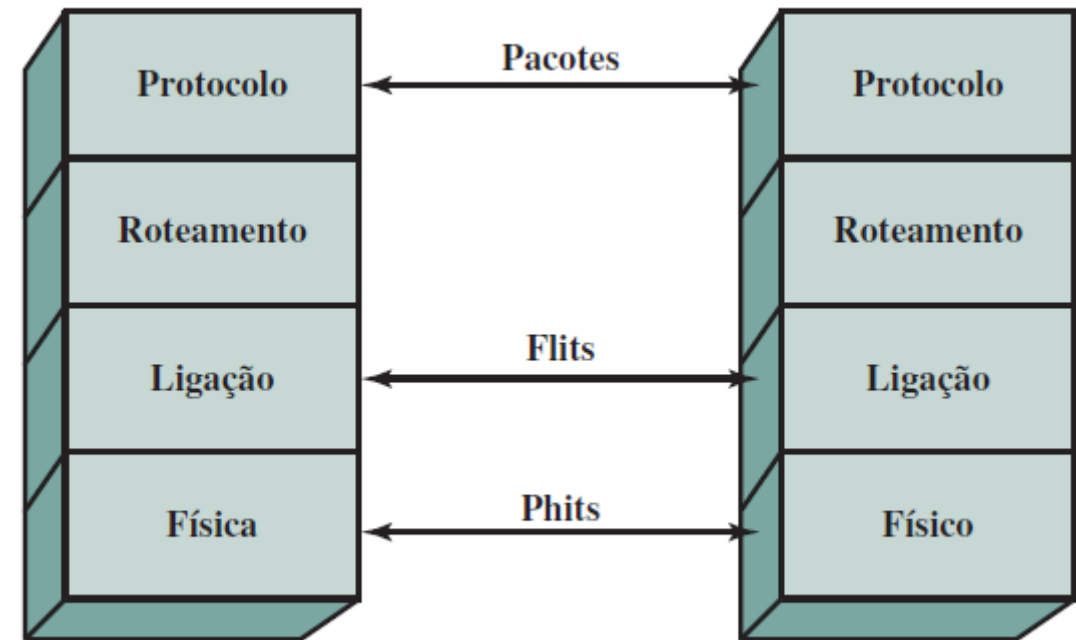
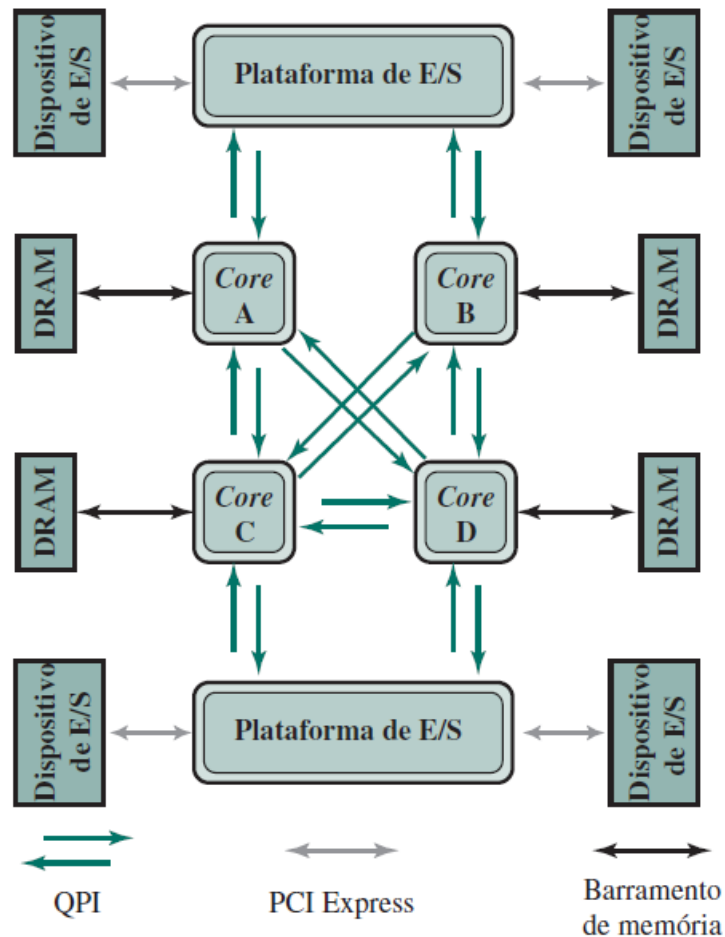
A principal razão que conduz essa mudança foram as restrições elétricas encontradas com o aumento da frequência dos barramentos sincronizados largos. Em taxas cada vez mais altas, tornou-se cada vez mais difícil realizar as funções de sincronização e arbitração em tempo hábil. Além do mais, com o advento dos chips multicore, com diversos processadores e memória significativa em um único chip, percebeu-se que o uso de um barramento compartilhado convencional no mesmo chip aumentou as dificuldades da elevação da taxa de dados e da redução da latência do barramento para acompanhar o processador.

Em comparação com o barramento compartilhado, a interconexão ponto a ponto tem menor latência, maior taxa de dados e melhor escalabilidade.

Barramento X Ponto a Ponto

Característica	Barramento	Ponto a ponto
Meio de comunicação	Compartilhado	Dedicado
Número de dispositivos	Vários	Normalmente dois por link
Disputa pelo meio	Sim	Muito menor
Paralelismo	Limitado	Maior
Escalabilidade	Menor	Maior
Complexidade	Menor	Maior
Desempenho	Pode ser limitado	Geralmente maior

Intel QuickPath Interconnect (QPI)



Flit = unidade lógica de transferência do protocolo.
Phit = unidade física de transferência pelo link.

Peripheral Component Interconnect (PCI)

O barramento PCI é um barramento de grande largura de banda, independente de processador, que pode funcionar como uma unidade intermediária ou barramento de periféricos.

Em comparação com outras especificações de barramento comuns, a PCI oferece melhor desempenho de sistema para subsistemas de E/S de alta velocidade (por exemplo, adaptadores gráficos de vídeo, controladores de interface de rede, controladores de disco e assim por diante).

PCI: Interconexão barramento.

PCI Express (PCIe): Interconexão ponto a ponto.

PCI: Barramento compartilhado

Versão	Ano	Frequência	Barramento	Largura de banda máxima	Tensão	Aplicações típicas	Principais novidades
PCI 1.0	1992	33 MHz	32 bits	133 MB/s	5 V	Primeiras placas de vídeo, som, rede	Primeira versão do padrão PCI, desenvolvida pela Intel.
PCI 2.0	1993	33 MHz	32 bits	133 MB/s	5 V	PCs comerciais	Melhorias na especificação e maior adoção pelos fabricantes.
PCI 2.1	1995	33/66 MHz	32 ou 64 bits	até 533 MB/s	3,3 V e 5 V	Servidores e workstations	Introduziu operação a 66 MHz e suporte a 64 bits.
PCI 2.2	1998	33/66 MHz	32 ou 64 bits	até 533 MB/s	3,3 V	PCs e servidores	Melhor compatibilidade elétrica e Plug and Play.
PCI 2.3	2002	33/66 MHz	32 ou 64 bits	até 533 MB/s	3,3 V	Equipamentos industriais e servidores	Removeu suporte obrigatório a dispositivos de 5 V.
PCI 3.0	2004	33/66 MHz	32 ou 64 bits	até 533 MB/s	3,3 V	Sistemas legados	Última revisão importante antes da substituição pelo PCI Express.

PCIe: Conexão ponto-a-ponto

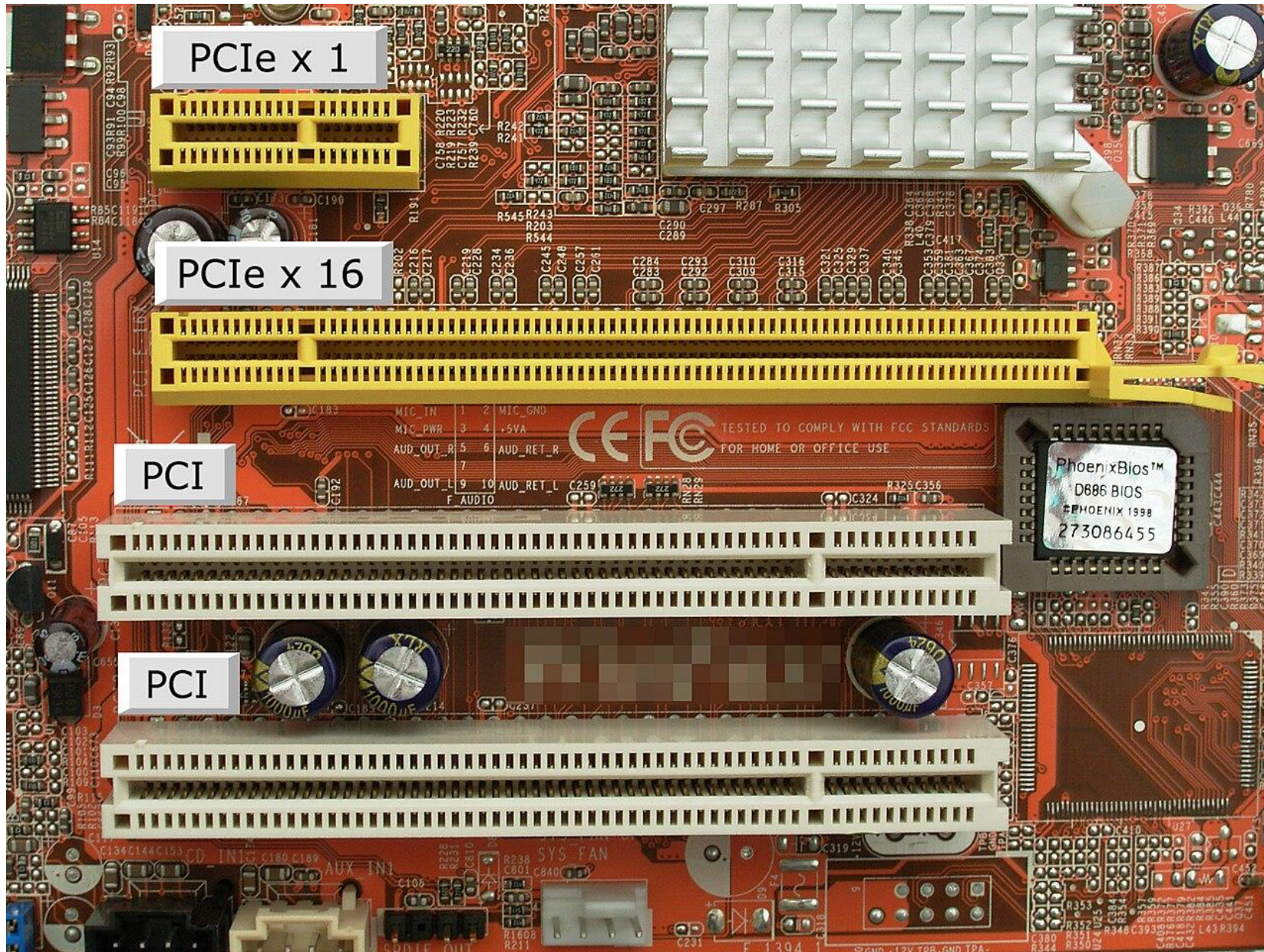
Versão	Ano de lançamento	Largura de banda por lane (cada direção)	Largura de banda máxima (x16, bidirecional)	Codificação	Aplicações típicas	Principais novidades
PCIe 1.0	2003	250 MB/s	8 GB/s	8b/10b	Placas de vídeo, placas de rede Gigabit, placas de som	Substituiu os barramentos PCI e AGP.
PCIe 2.0	2007	500 MB/s	16 GB/s	8b/10b	GPUs mais rápidas, SSDs iniciais, RAID	Dobrou a largura de banda da versão 1.0.
PCIe 3.0	2010	985 MB/s	31,5 GB/s	128b/130b	SSD NVMe, GPUs modernas, placas de rede 10 GbE	Melhor eficiência graças à nova codificação.
PCIe 4.0	2017	1,969 GB/s	63 GB/s	128b/130b	SSDs NVMe Gen4, GPUs de alto desempenho, servidores	Dobrou novamente a largura de banda.
PCIe 5.0	2019	3,938 GB/s	126 GB/s	128b/130b	GPUs atuais, SSDs Gen5, IA, redes de 100 GbE	Muito utilizado em servidores e workstations.
PCIe 6.0	2022	7,877 GB/s	252 GB/s	PAM4 + FEC	IA, HPC, datacenters, redes 400 GbE	Introduziu sinalização PAM4 e correção de erros (FEC).
PCIe 7.0	2025	15,75 GB/s	504 GB/s	PAM4 + FEC	IA de larga escala, HPC, computação em nuvem	Dobra novamente a largura de banda do PCIe 6.0.

PCIe x 1

PCIe x 16

PCI

PCI



Exercício Prático

Descobrir quantas conexões PCI e PCIe existem na placa mãe do seu computador (e quais as versões).



Placa Mãe X870-Plus Asus TUF Gaming

Que categorias gerais de funções são especificadas pelas instruções do computador?

- ❑ **Processamento de Dados (Data Processing)** Realizam operações sobre os dados, como operações aritméticas e lógicas. Exemplos: soma, subtração, multiplicação, divisão, operações AND, OR e NOT.
- ❑ **Armazenamento de Dados (Data Storage)** Transferem dados entre registradores e memória. Exemplos: LOAD (carregar um dado da memória para um registrador) e STORE (armazenar um registrador na memória).
- ❑ **Movimentação de Dados (Data Movement)** Movem dados entre o computador e dispositivos de entrada e saída (E/S) ou entre diferentes componentes internos. Exemplos: leitura de teclado, escrita em disco e comunicação com dispositivos.
- ❑ **Controle (Control)** Alteram a sequência de execução do programa. Exemplos: desvios condicionais (if), desvios incondicionais (goto), chamadas e retornos de sub-rotinas.

Liste e defina os estados possíveis que definem a execução de uma instrução.

- ☐ **Busca da Instrução (Fetch)** O processador busca na memória a próxima instrução a ser executada.
- ☐ **Busca Indireta (Indirect)** Caso o endereço seja indireto, o processador realiza uma busca adicional para obter o endereço efetivo do operando.
- ☐ **Execução (Execute)** A instrução é executada conforme sua operação, podendo envolver cálculos, movimentação de dados ou alteração do fluxo do programa.
- ☐ **Interrupção (Interrupt)** Antes de iniciar a próxima instrução, o processador verifica se existe alguma interrupção pendente. Caso exista, salva o contexto atual e transfere o controle para a rotina de tratamento da interrupção.

Que tipos de transferências a estrutura de interconexão de um computador (por exemplo, barramento) precisa aceitar?

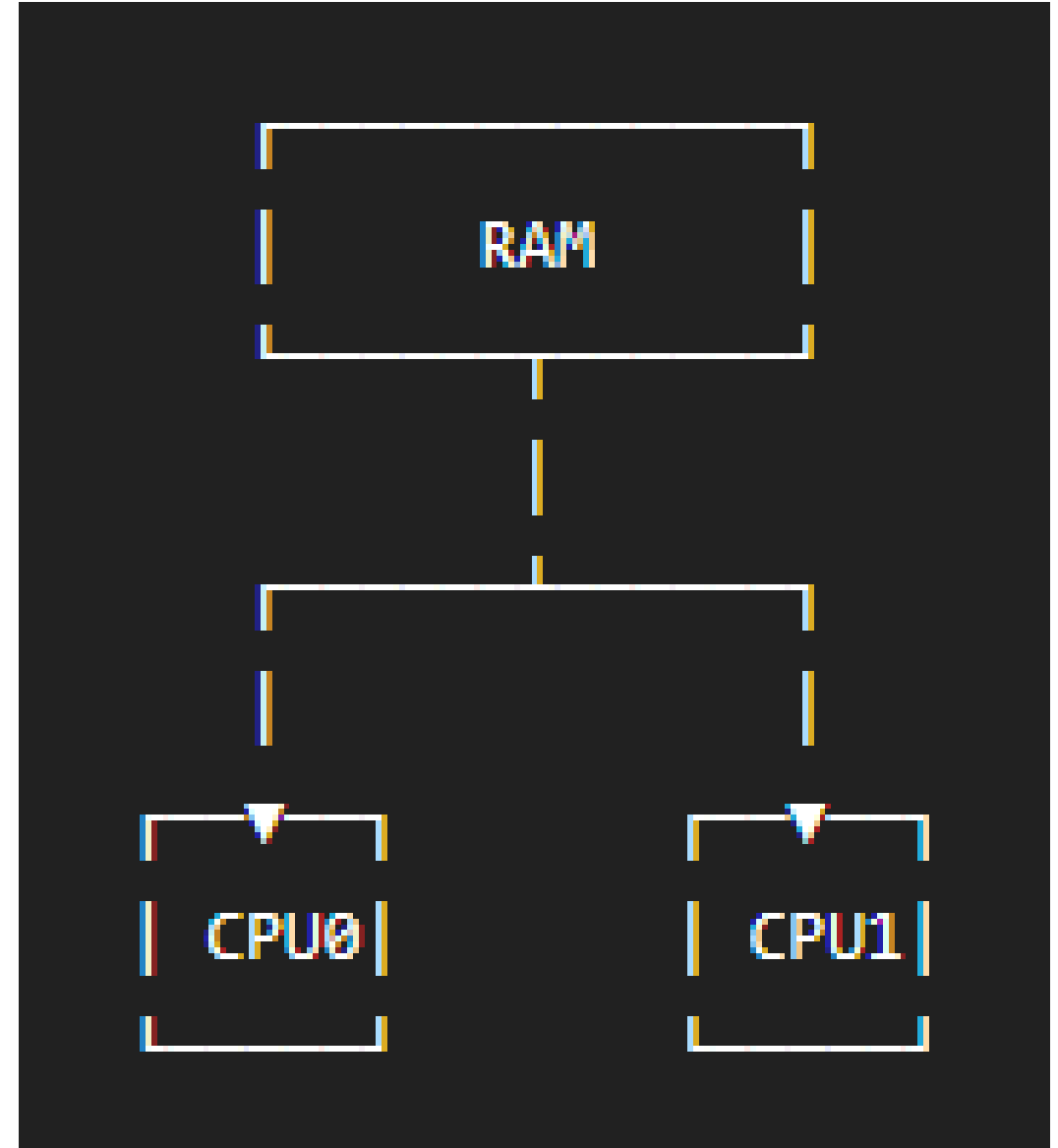
- ❑ **Memória → Processador:** Leitura de dados ou instruções da memória principal.
- ❑ **Processador → Memória:** Escrita de dados na memória principal.
- ❑ **Entrada/Saída ↔ Processador:** Comunicação entre dispositivos periféricos e o processador.
- ❑ **Entrada/Saída ↔ Memória (DMA – Direct Memory Access):** Transferência direta entre dispositivos de E/S e a memória principal, sem participação contínua do processador.

Memória Principal

UMA = Uniform Memory Access, ou Acesso Uniforme à Memória.

Todos os processadores possuem acesso a todas as partes da memória principal (leituras e escritas).

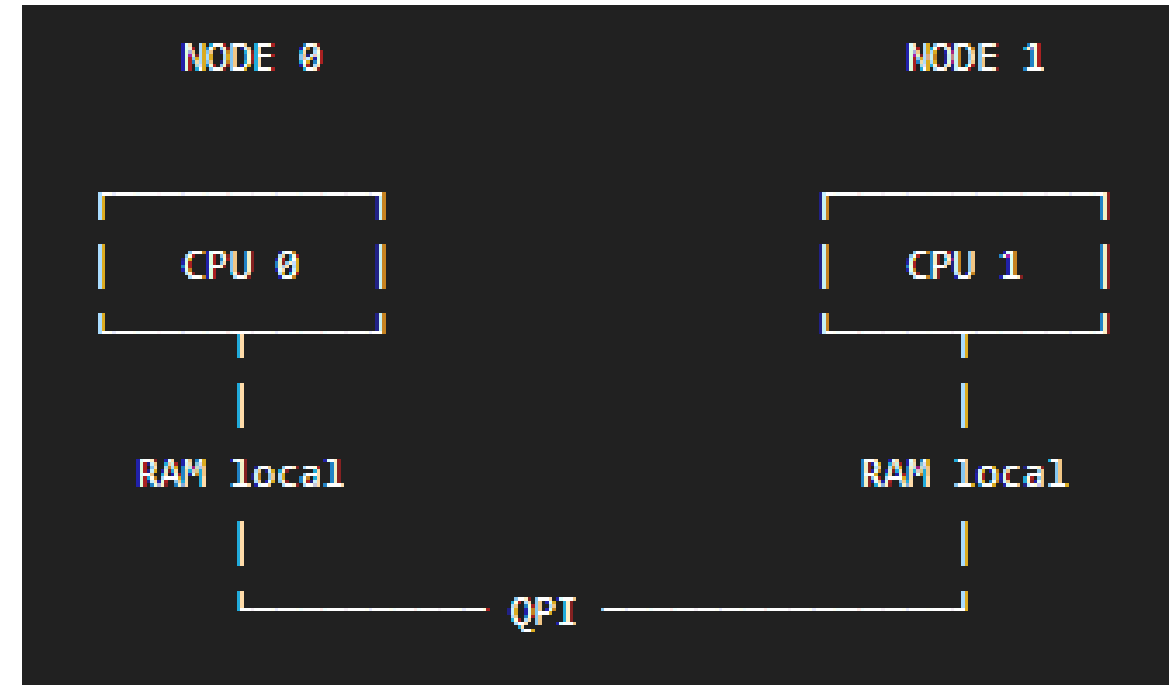
O tempo de acesso à memória de um processador para todas as regiões da memória é o mesmo. Os tempos de acesso de processadores diferentes são os mesmos.



NUMA = Non-Uniform Memory Access, ou Acesso Não Uniforme à Memória.

Aqui, cada processador (ou nó) possui uma **memória local**, mas também pode acessar a memória pertencente a outro processador.

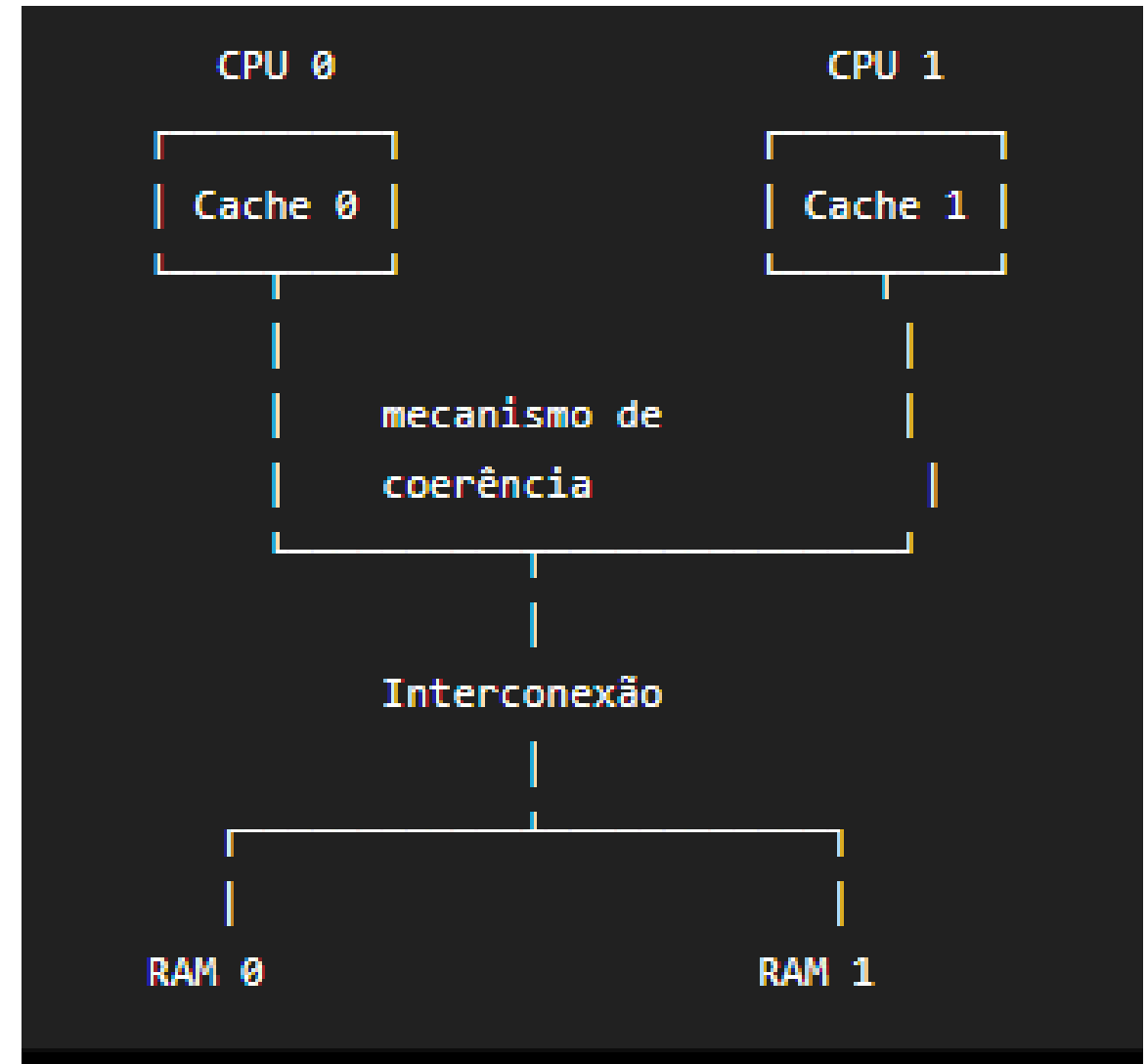
Porém, acessar a memória local é normalmente **mais rápido** do que acessar a memória remota.



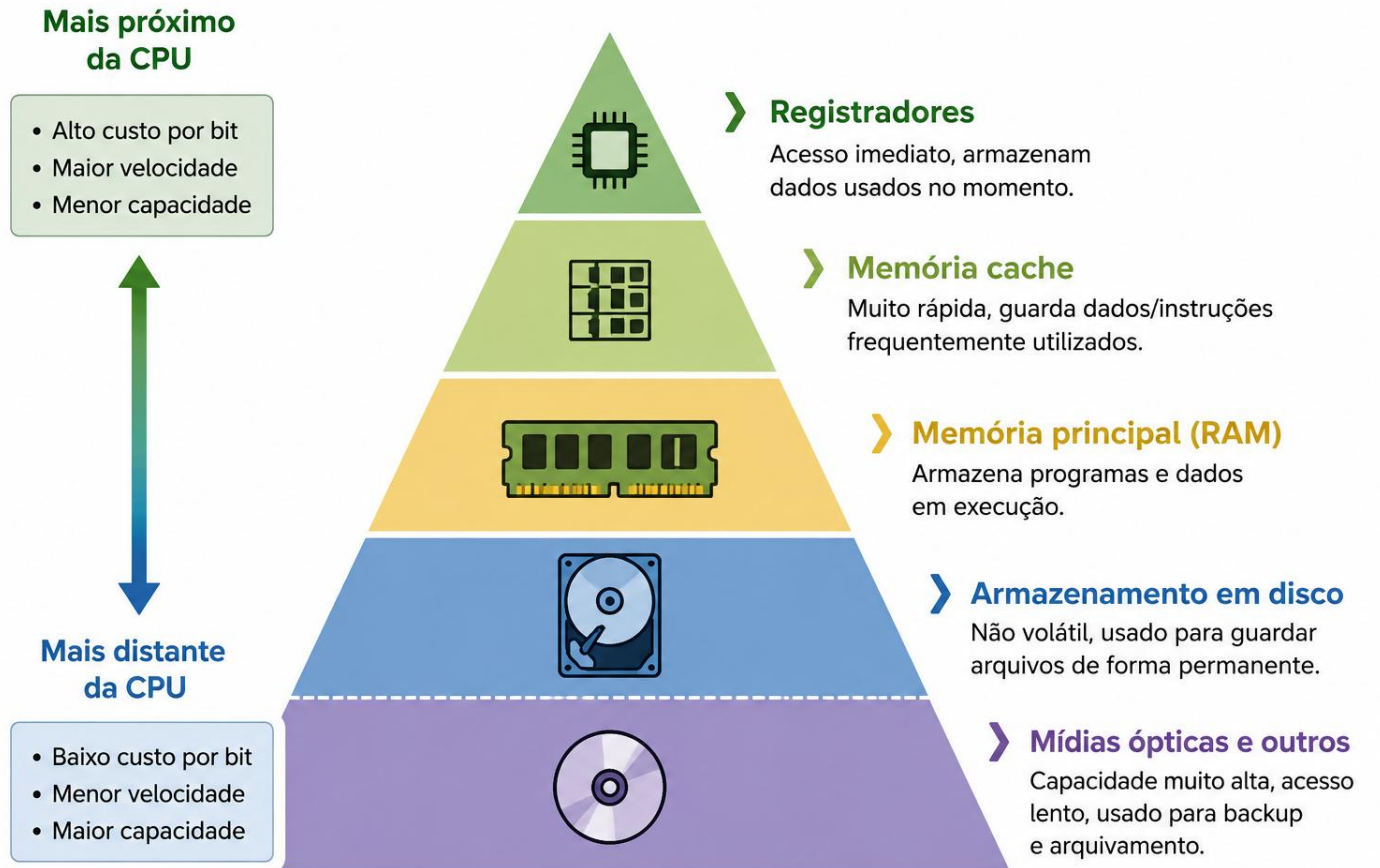
CC-NUMA (Cache Coherent NUMA)

O **CC-NUMA** adiciona mecanismos para garantir que as caches das diferentes CPUs mantenham uma visão **coerente** dos dados compartilhados.

Se uma CPU modificar um dado que está sendo utilizado por outra CPU, o sistema possui mecanismos para **manter a coerência das cópias em cache**.



Hierarquia de Memória



Memória Principal

A memória principal é a memória indispensável para o funcionamento do computador, pois é onde ficam os programas e dados a serem executados/processados pelo processador.

RAM (Random Access Memory): Todas as informações do computador passam por ela e só permanecem lá enquanto houver energia elétrica, ou seja, trata-se de uma **memória volátil**.

DRAM (Dynamic Random Access Memory)

O termo dinâmico indica que a memória deve ser constantemente atualizada, ou perderá seu conteúdo. Normalmente é utilizada para a memória principal em dispositivos de informática.

SDRAM (Synchronous Dynamic Random-Access Memory)

Synchronous (Síncrona): trabalha sincronizada com o **clock** do sistema.

Dynamic (Dinâmica): os bits são armazenados em células que precisam ser periodicamente **atualizadas (refresh)**.

Random Access (Acesso aleatório): é possível acessar diretamente uma posição de memória, sem precisar percorrer as posições anteriores.

DDR SDRAM (Double Data Rate SDRAM)

Exemplo:

DDR4-3200 = 3200 MT/s (milhões de transferências por segundo)

Não significa que o clock físico da memória seja 3200 MHz. O clock base é aproximadamente **1600 MHz**, com duas transferências por ciclo.

DDR SDRAM – Largura de Banda

Largura de banda = taxa de transferência × largura do barramento

Exemplo:

$$\begin{aligned}\text{Largura de banda DDR4-3200} &= 3200 \text{ MT/s} \times 64 \text{ bits} \\ &= 3.200.000.000 \times 8 \text{ bytes} \\ &= 25.600.000.000 \text{ bytes/s} \\ &= 25,6 \text{ GB/s}\end{aligned}$$

DDR SDRAM - Modelos

Característica	DDR	DDR2	DDR3	DDR4	DDR5
Ano de introdução	2000	2003	2007	2014	2020/2021
Nome completo	DDR SDRAM	DDR2 SDRAM	DDR3 SDRAM	DDR4 SDRAM	DDR5 SDRAM
Pré-busca (Prefetch)	2n	4n	8n	8n	16n
Taxa de transferência típica/JEDEC	200–400 MT/s	400–1066 MT/s	800–2133 MT/s	1600–3200 MT/s	3200 MT/s em diante*
Clock efetivo	100–200 MHz	200–533 MHz	400–1066 MHz	800–1600 MHz	1600 MHz em diante*
Tensão típica	2,5 V	1,8 V	1,5 V	1,2 V	1,1 V
Taxa de transferência por ciclo	2	2	2	2	2
Largura tradicional do módulo	64 bits	64 bits	64 bits	64 bits	64 bits, divididos em 2 × 32 bits
Eficiência energética	Baixa	Melhor	Melhor	Alta	Mais alta
Densidade	Baixa	Maior	Maior	Muito maior	Muito maior
ECC	Dependente do módulo	Dependente do módulo	Dependente do módulo	Dependente do módulo	Suporte à correção interna (on-die ECC)
Uso atual	Obsoleta	Obsoleta	Legado	Ainda utilizada	Principal geração atual

ROM (Read Only Memory)

É um tipo de memória principal. As informações dessa memória, não podem ser apagadas, pois seus dados já vêm gravados de fábrica. São informações preestabelecidas durante a fabricação, como, por exemplo, as características do hardware.

PROM (Programmable Read-Only Memory)

É uma memória que pode ser **programada uma única vez**.

É uma memória **OTP (One-Time Programmable)**.

Exemplo: Controladores industriais e equipamentos eletrônicos antigos

EPROM (Erasable Programmable Read-Only Memory)

Diferentemente da PROM, a EPROM pode ser **apagada e programada novamente**.

A EPROM é apagada utilizando luz ultravioleta (UV).

Exemplo: Um exemplo clássico são placas eletrônicas industriais.

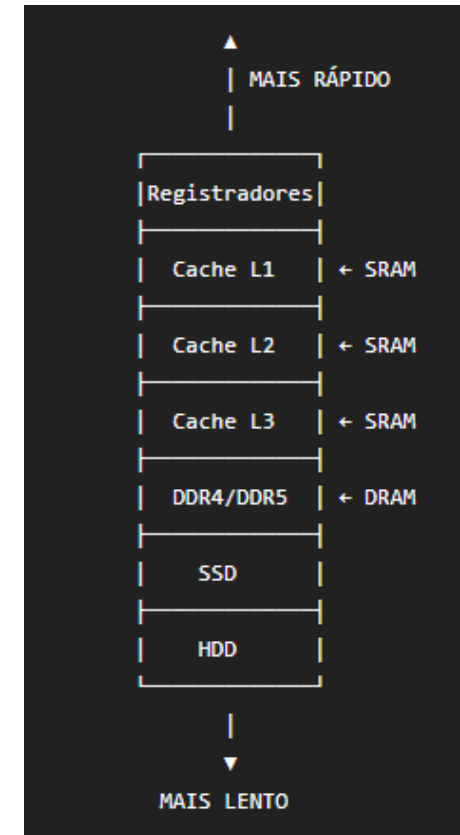
EEPROM (Electrically Erasable Programmable Read-Only Memory)

A EEPROM pode ser apagada eletricamente.

Exemplo: Microcontroladores, impressoras, equipamentos industriais, BIOS/firmware de alguns dispositivos

SRAM (Static Random Access Memory)

A palavra **Static (estática)** significa que, enquanto houver alimentação elétrica, o circuito consegue manter o estado armazenado sem precisar ficar sendo atualizado periodicamente.



SRAM × SDRAM



	SRAM	SDRAM/DDR
Tipo	Static RAM	Dynamic RAM
Volátil	Sim	Sim
Refresh	Não	Sim
Velocidade	Muito alta	Alta
Densidade	Baixa	Alta
Custo/bit	Alto	Baixo
Uso	Cache	RAM principal
Célula típica	~6 transistores	1 transistor + capacitor

Memória CMOS (Complementary Metal-Oxide-Semiconductor)

CMOS não é exatamente um tipo de memória. É uma **tecnologia de fabricação de circuitos eletrônicos** que utiliza transistores MOSFET complementares.

Porém, na arquitetura dos PCs, o termo "**memória CMOS**" passou a ser usado para designar uma pequena região de memória utilizada para armazenar **configurações do firmware e informações do sistema**, como:

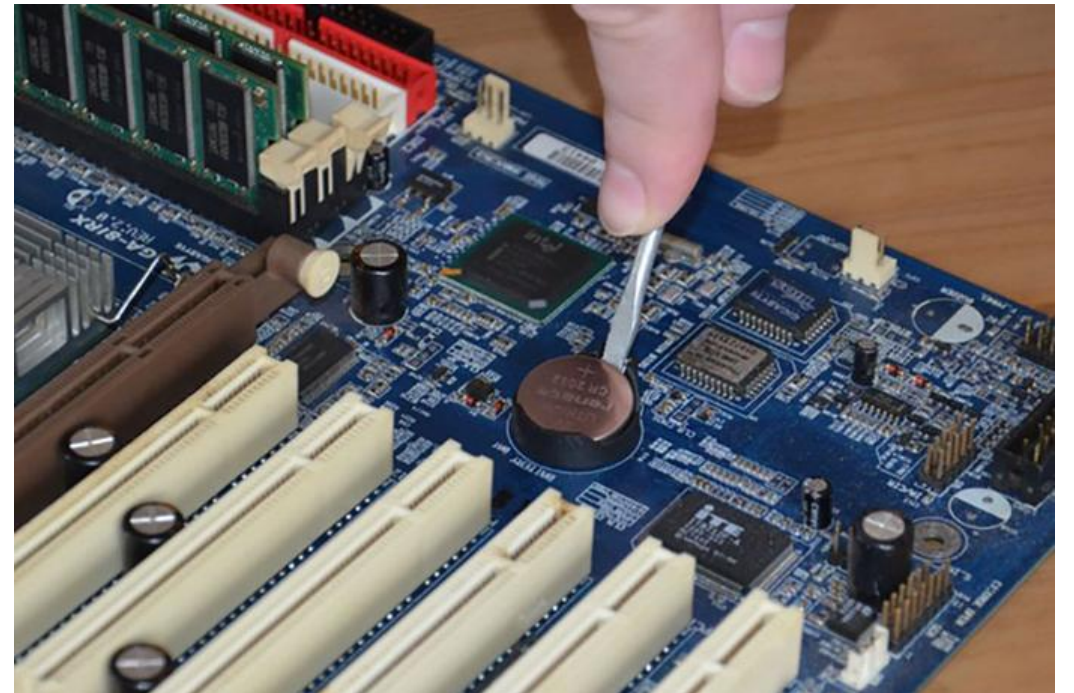
- ☐ data e hora;
- ☐ configuração de dispositivos;
- ☐ parâmetros de inicialização;
- ☐ algumas configurações do BIOS/UEFI.

Historicamente, essa memória era implementada usando tecnologia CMOS e precisava de uma pequena alimentação elétrica para preservar os dados.

Memória CMOS (Complementary Metal-Oxide-Semiconductor)

A famosa bateria **CR2032** da placa-mãe está relacionada principalmente ao **RTC (Real-Time Clock)** e, nas plataformas tradicionais, ajudava a manter a memória CMOS.

Quando o computador era desligado, a bateria fornecia energia suficiente para manter o **RTC** e a pequena memória CMOS.



Memória Cache

Princípio da Localidade Temporal

Um dado acessado recentemente tem mais chances de ser usado novamente do que um dado usado há mais tempo. Isso ocorre porque as variáveis de um programa tendem a ser acessadas diversas vezes durante a execução de um programa, e as instruções utilizam muitos comandos de repetição (laços) e subprogramas, fazendo com que as instruções sejam acessadas repetidamente.

```
// C

for (int i = 0; i < 1000000; i++) {
    soma += valor;
}
```

A variável `valor` é utilizada repetidamente.
O cache tenta manter esse dado próximo da CPU.

Princípio da Localidade Espacial

Há uma maior probabilidade de acesso para dados e instruções em endereços próximos àqueles acessados recentemente. Isso ocorre porque os programas são sequenciais e usam laços. Quando uma instrução é acessada, a instrução com maior probabilidade de ser executada na sequência é a instrução logo a seguir dela. Para as variáveis a ideia é a mesma, pois variáveis de um mesmo programa são armazenadas próximas umas das outras, vetores e matrizes são armazenados em sequência de acordo com seus índices.

```
// C

int vetor[1000];

for (int i = 0; i < 1000; i++) {
    printf("%d", vetor[i]);
}
```

Os elementos estão armazenados em posições próximas.

O cache aproveita isso trazendo **um bloco de memória**, e não apenas um único byte.

Memória Cache

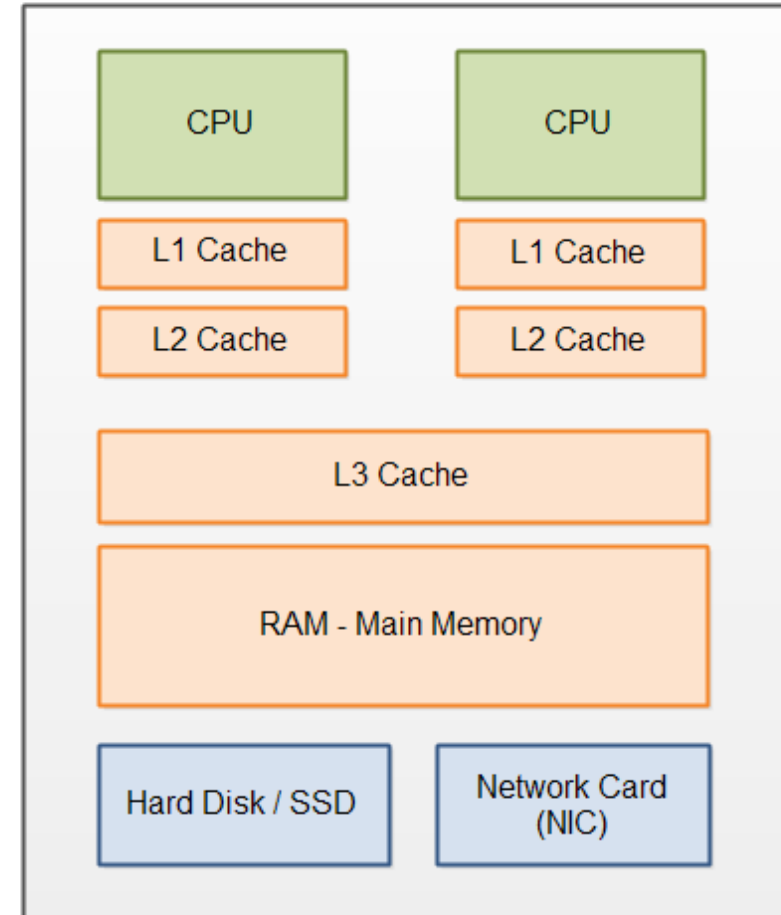
A CPU moderna é extremamente rápida, enquanto a RAM é relativamente lenta. Se a CPU tivesse que esperar a RAM a cada acesso, grande parte do potencial de processamento seria desperdiçada.

O cache funciona como uma **memória intermediária muito rápida**, armazenando dados e instruções que a CPU provavelmente utilizará novamente.

Cache L1

O **L1 (Level 1)** é o cache mais próximo do núcleo da CPU.

- ❑ extremamente rápido;
- ❑ pequena capacidade;
- ❑ normalmente separado por núcleo;
- ❑ geralmente dividido em **L1 de instruções (L1I)** e **L1 de dados (L1D)**.

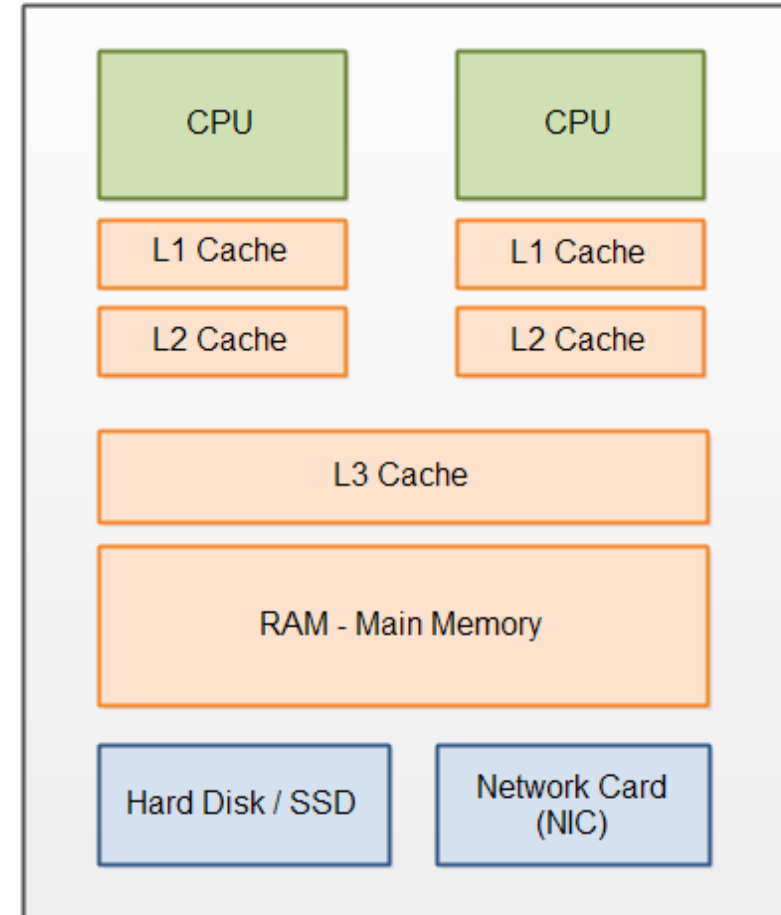


Cache L2 e L3

O **L2 (Level 2)** é normalmente maior que L1, mas apresenta uma latência maior.

O **L3 (Level 3)** normalmente é:

- ❑ maior;
- ❑ mais lento que L1 e L2;
- ❑ compartilhado entre vários núcleos.



AMD Ryzen 7 5800x

CPU Caches Mainboard Memory SPD Graphics Bench About

Processor

Name	AMD Ryzen 7 5800X		
Code Name	Vermeer	Max TDP	105.0 W
Package	Socket AM4 (1331)		
Technology	7 nm	Core Voltage	1.408 V

Specification

AMD Ryzen 7 5800X 8-Core Processor			
Family	F	Model	1
Ext. Family	19	Ext. Model	21
Instructions	MMX(+), SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, SSE4A, x86-64, AES, AVX, AVX2, FMA3, SHA		

Clocks (Core #0)

Core Speed	3555.3 MHz
Multiplier	x 36.0
Bus Speed	98.8 MHz
Rated FSB	

Caches

L1 Data	8 x 32 KBytes	8-way
L1 Inst.	8 x 32 KBytes	8-way
Level 2	8 x 512 KBytes	8-way
Level 3	32 MBytes	16-way

Selection **Socket #1** Cores **8** Threads **16**

TUF GAMING

CPU Caches Mainboard Memory SPD Graphics Bench About

L1 D-Cache

Size	32 KBytes	x 8
Descriptor	8-way set associative, 64-byte line size	

L1 I-Cache

Size	32 KBytes	x 8
Descriptor	8-way set associative, 64-byte line size	

L2 Cache

Size	512 KBytes	x 8
Descriptor	8-way set associative, 64-byte line size	

L3 Cache

Size	32 MBytes	
Descriptor	16-way set associative, 64-byte line size	

Mapeamento de Cache

A RAM possui muito mais posições do que o cache. Onde colocar cada bloco da RAM dentro do cache?

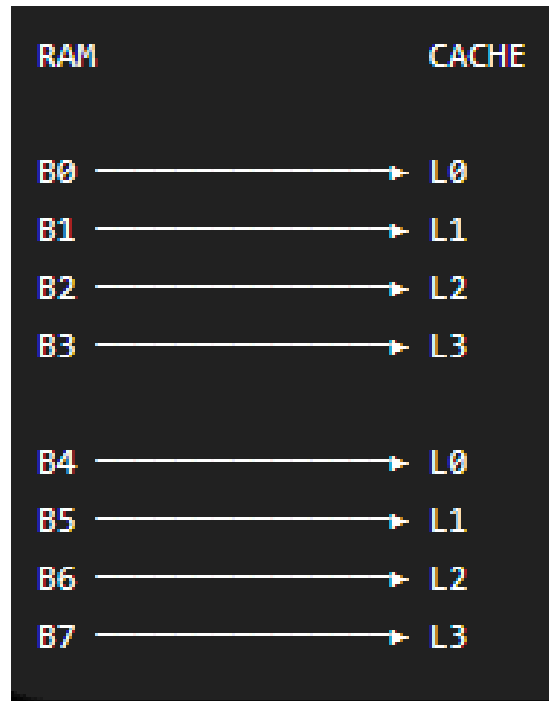
A RAM é dividida em blocos e o Cache em linhas.

Os três métodos clássicos são:

- Mapeamento direto
- Mapeamento associativo completo
- Mapeamento associativo por conjuntos

Mapeamento Direto

Cada bloco da memória principal é associado a uma única linha específica do cache, determinada pelo índice do bloco módulo o número de linhas do cache.



$$B0 \bmod 4 = 0 \rightarrow L0$$

$$B1 \bmod 4 = 1 \rightarrow L1$$

$$B2 \bmod 4 = 2 \rightarrow L2$$

$$B3 \bmod 4 = 3 \rightarrow L3$$

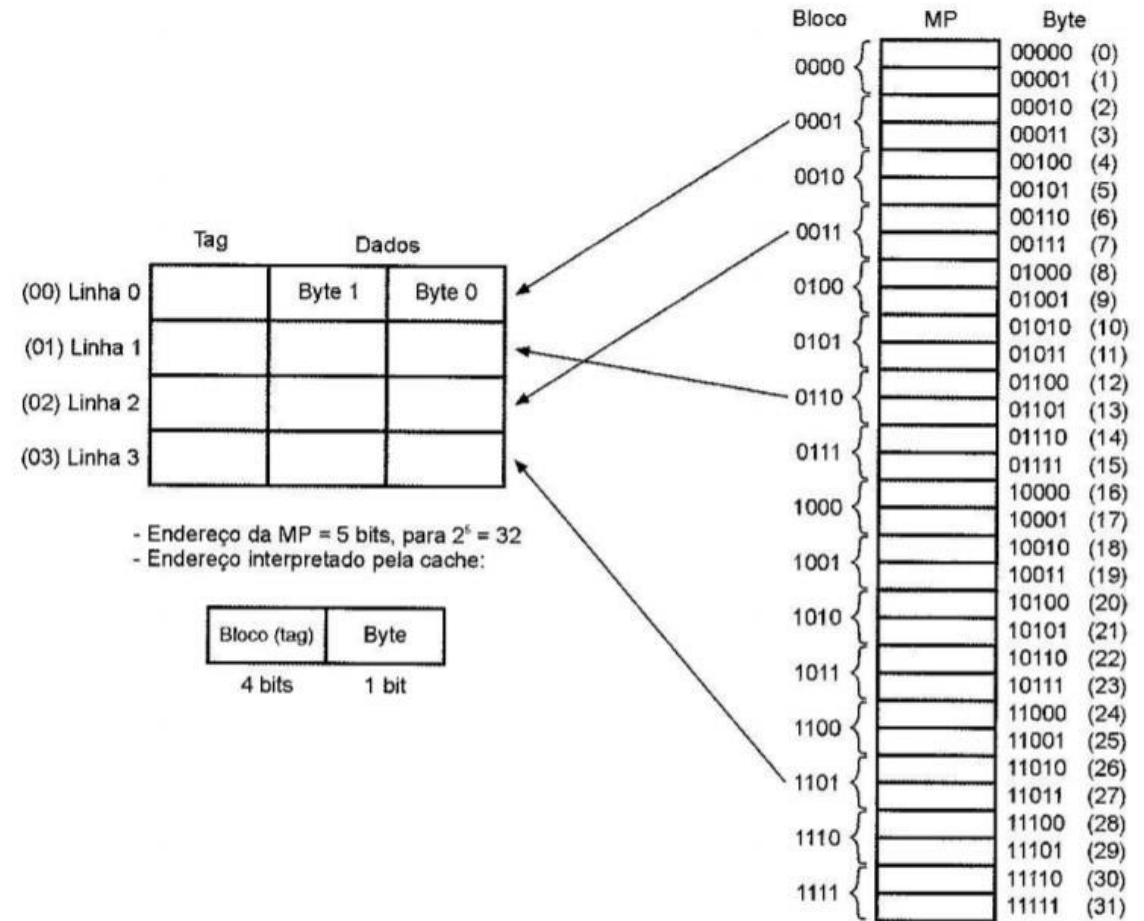
$$B4 \bmod 4 = 0 \rightarrow L0$$

$$B5 \bmod 4 = 1 \rightarrow L1$$

Linha = número do bloco mod número de linhas do cache.

Mapeamento associativo completo

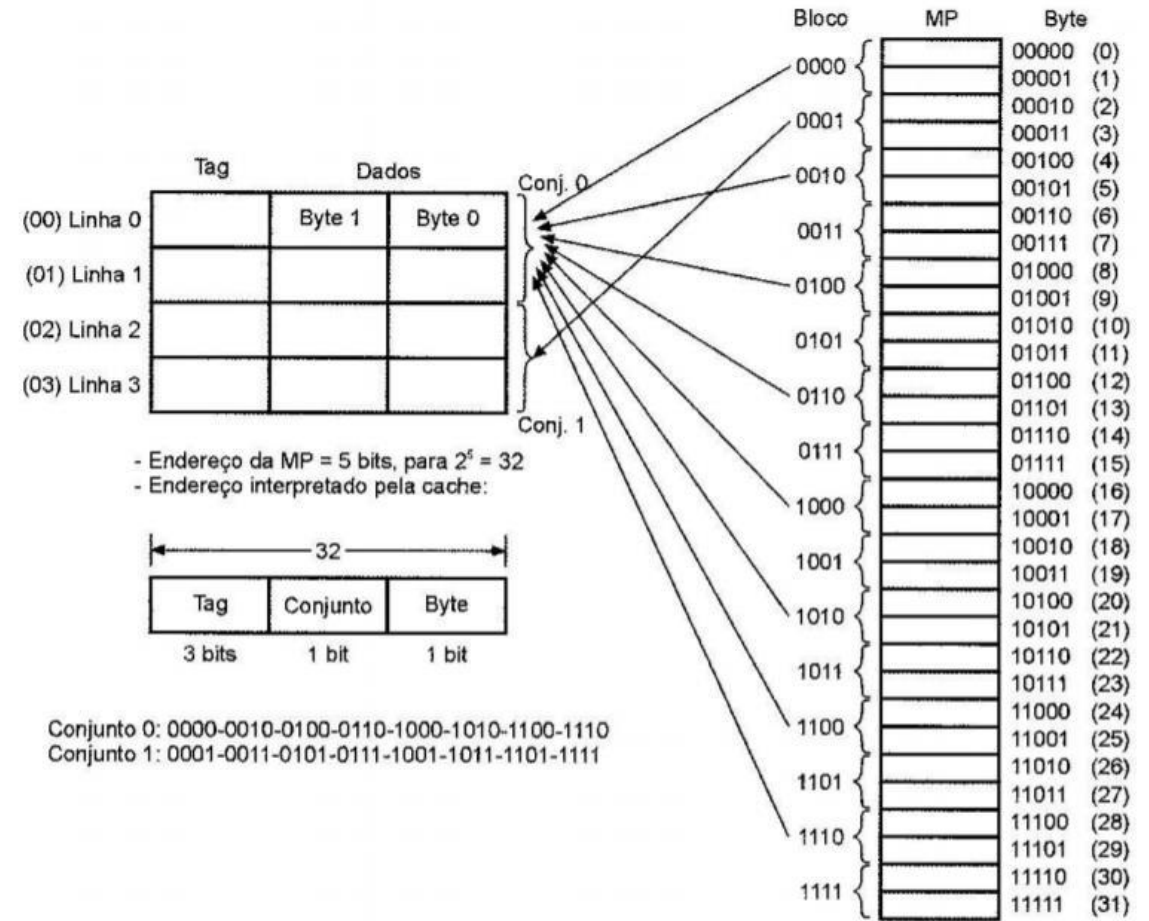
Cada bloco da memória principal pode ser armazenado em qualquer linha disponível do cache, sendo necessário comparar a tag do endereço com as tags das linhas para determinar se o bloco está presente.



Problema: Quando a CPU procura um bloco, ela precisa verificar **todas as linhas do cache**.

Mapeamento associativo por conjuntos

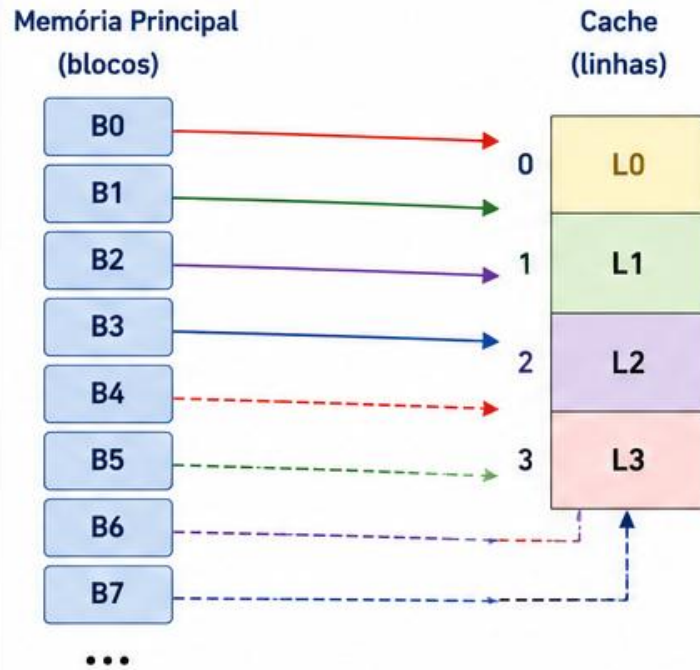
Cada bloco da memória principal é associado a um único conjunto do cache, determinado pelo número do bloco módulo o número de conjuntos, podendo ocupar qualquer linha dentro desse conjunto.



COMPARAÇÃO VISUAL DOS MAPEAMENTOS DE CACHE

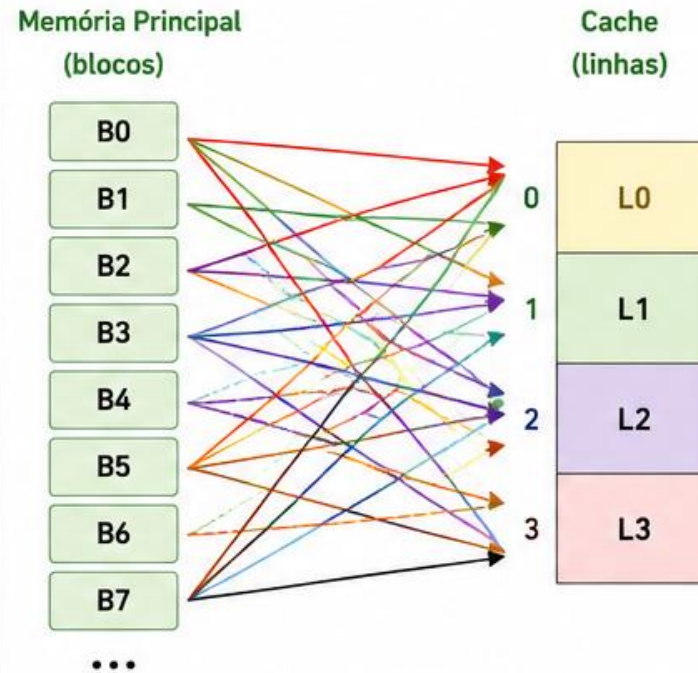
1. MAPEAMENTO DIRETO

Cada bloco da memória principal só pode ser armazenado em **UMA ÚNICA** linha do cache.



2. MAPEAMENTO ASSOCIATIVO COMPLETO

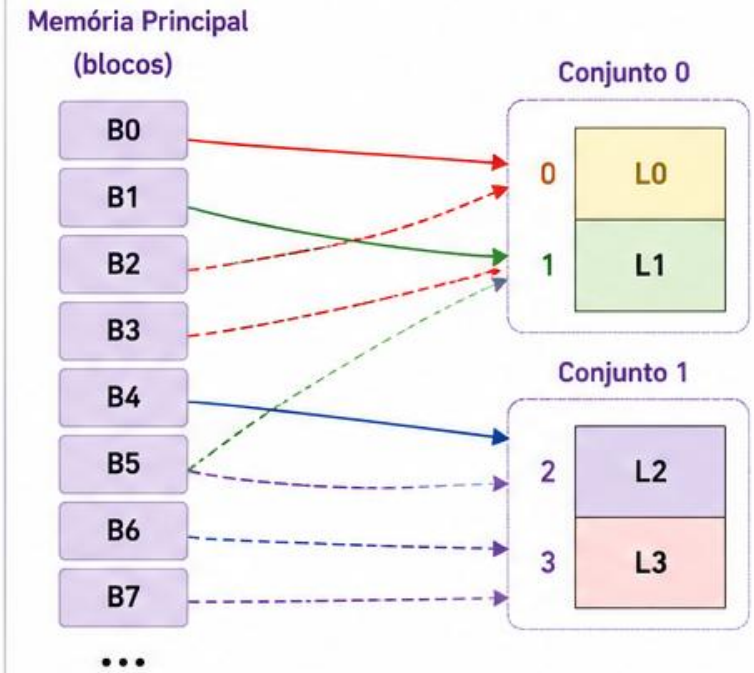
Qualquer bloco da memória principal pode ser armazenado em **QUALQUER** linha do cache.



Regra: não há restrição de posição.
O bloco pode ficar em qualquer linha do cache.

3. MAPEAMENTO ASSOCIATIVO POR CONJUNTOS

Cada bloco da memória principal é associado a **UM CONJUNTO** do cache, mas pode ocupar **QUALQUER** linha dentro desse conjunto.

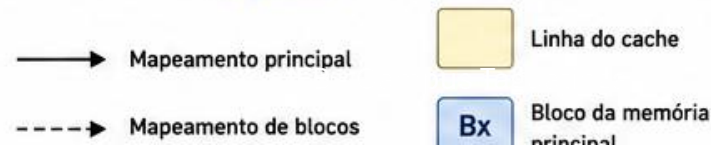


Regra: conjunto = número do bloco mod número de conjuntos
Ex.: com 2 conjuntos → pares vão para o conjunto 0
ímpares vão para o conjunto 1

Resumo das Regras

- 1 Direto: Bloco → **UMA ÚNICA LINHA**
- 2 Associativo completo: Bloco → **QUALQUER LINHA**

Legenda



Comparação Geral



Compilação e Interpretação

Conceitos

Linguagens de alto nível: próxima do homem

❑ Instrução de alto nível x ling. montagem: 1:N

Linguagens de baixo nível: próxima do hardware

❑ Linguagem de montagem x código objeto: 1:1

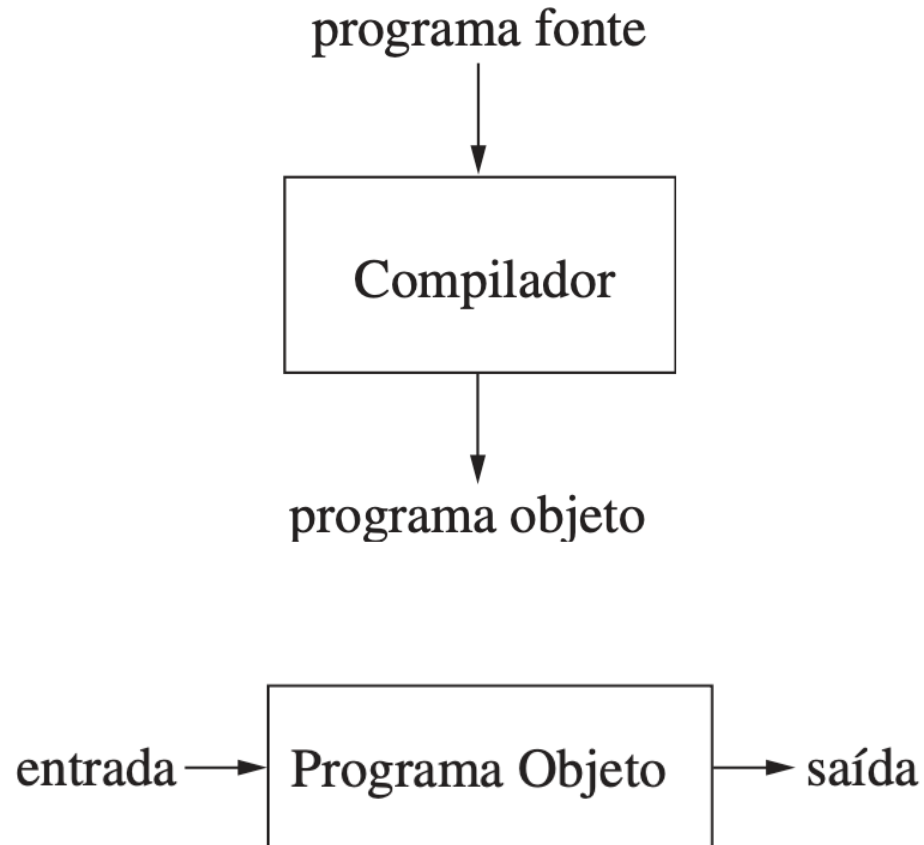
Código-objeto: binário

❑ Ligador (likekeditor): liga códigos objetos em tempo de compilação.

- EX: gcc *.c -o exec

❑ DLLs (Dynamic Link Library): carga em tempo de execução.

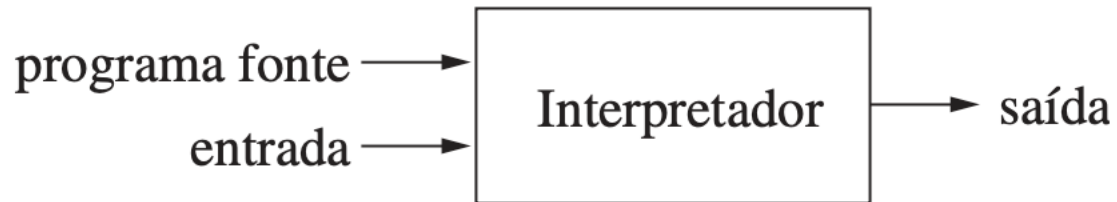
Processadores de Linguagem



Um **compilador** é um programa que recebe como entrada um programa em uma linguagem de programação - a linguagem fonte - e o traduz para um programa equivalente em outra linguagem - a **linguagem objeto**.

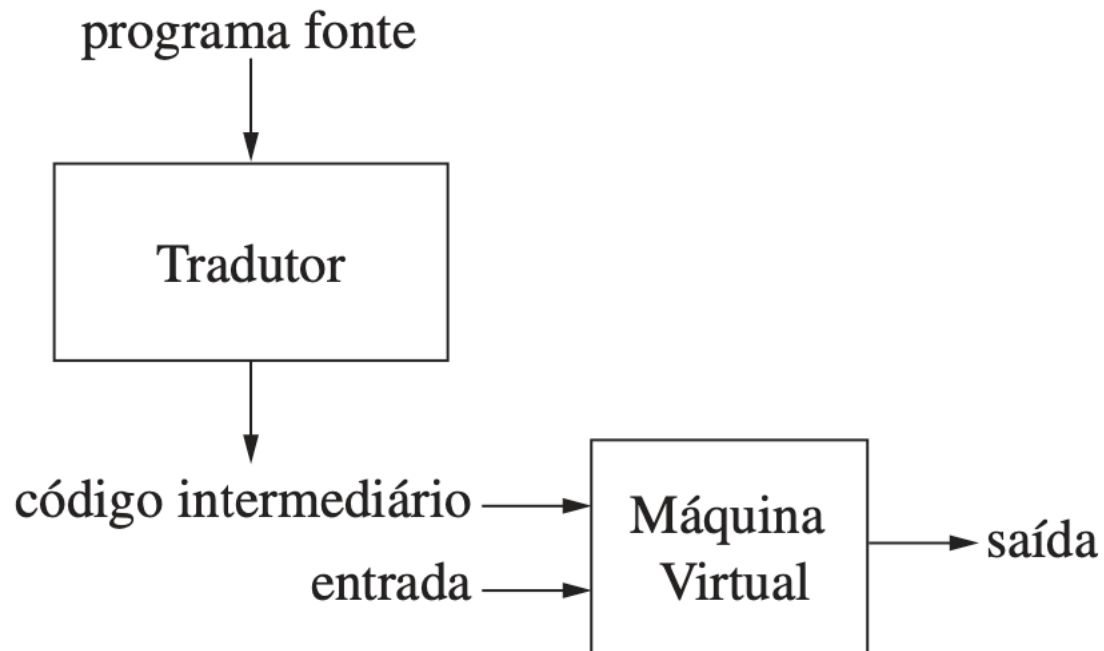
Se o programa objeto for um programa em uma linguagem de máquina executável, poderá ser chamado pelo usuário para processar entradas e produzir saída.

Processadores de Linguagem



Um **interpretador** é outro tipo comum de processador de linguagem. Em vez de produzir um programa objeto como resultado da tradução, um interpretador executa diretamente as operações especificadas no programa fonte sobre as entradas fornecidas pelo usuário.

Processadores de Linguagem

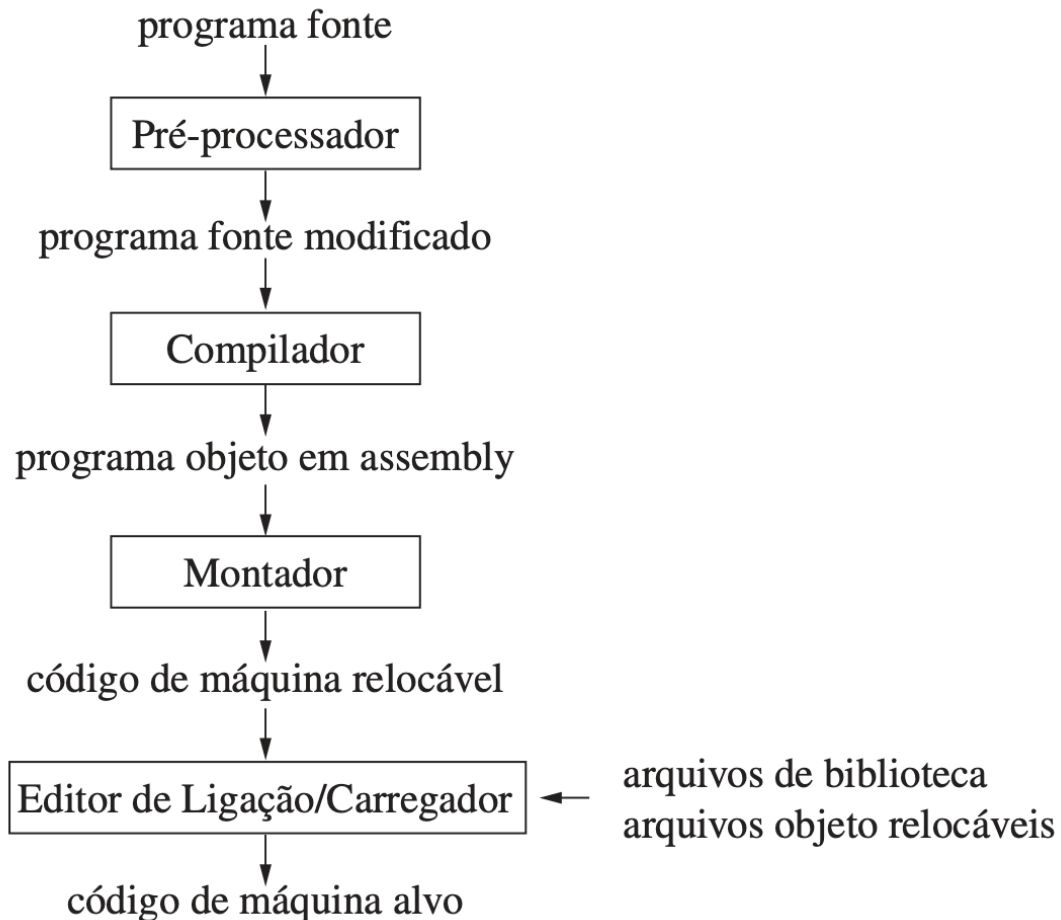


Os processadores da linguagem Java combinam compilação e interpretação.

Um programa fonte em Java pode ser primeiro compilado para uma forma **intermediária, chamada bytecodes**. Os bytecodes (ou códigos de bytes) são então interpretados por uma máquina virtual.

Como um benefício dessa combinação, os bytecodes compilados em uma máquina podem ser interpretados em outra máquina, talvez por meio de uma rede.

Compilação



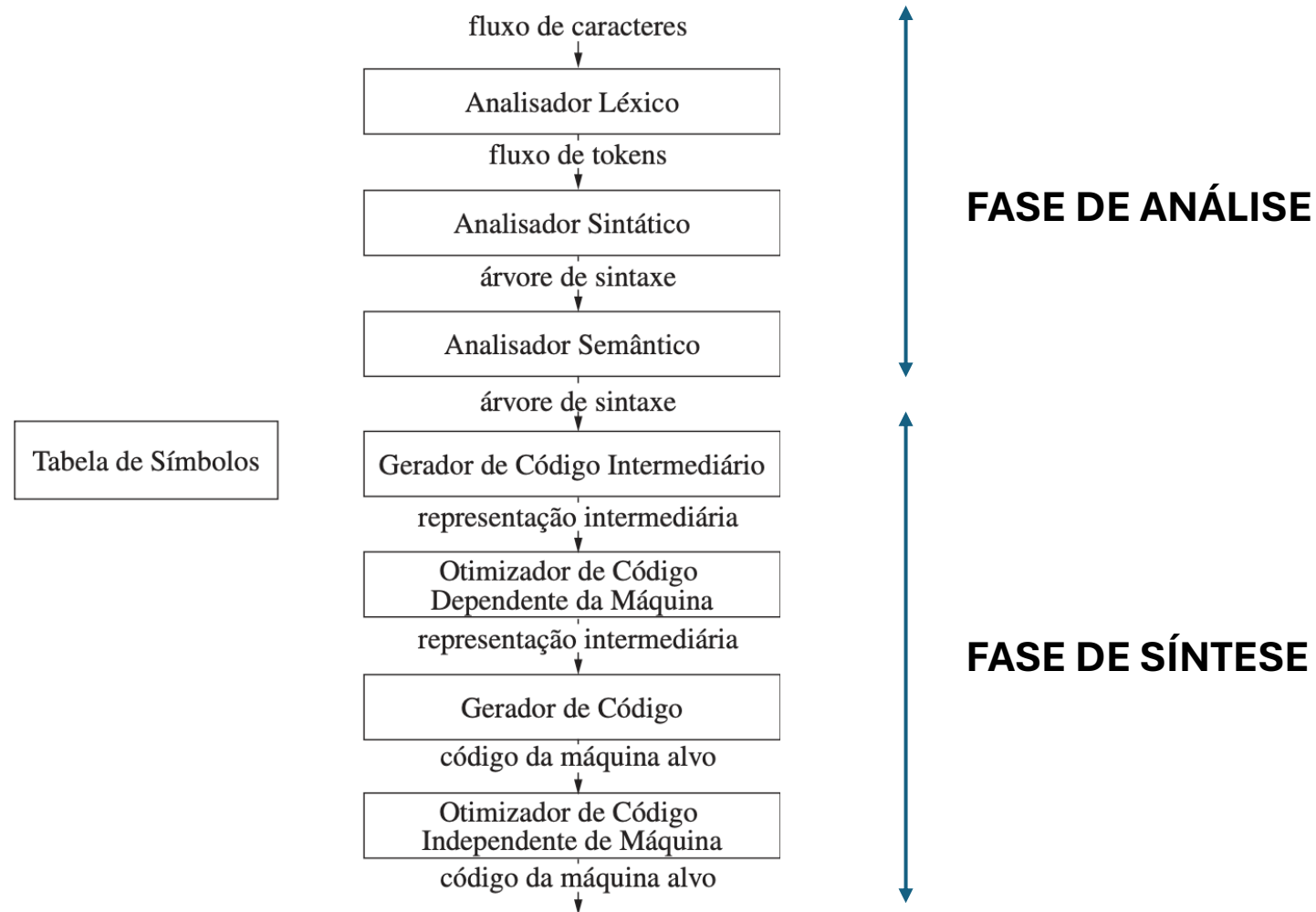
O **compilador** recebe na entrada o programa fonte modificado e pode produzir como saída um programa em uma linguagem simbólica, conhecida como assembly.

A linguagem simbólica é então processada por um programa chamado **montador (assembler)**, que produz código de máquina relocável como sua saída.

O **editor de ligação (linker)** resolve os endereços de memória externos, onde o código em um arquivo pode referir-se a uma localização em outro arquivo.

O **carregador (loader)** reúne então todos os arquivos objeto executáveis na memória para a execução.

Fases de um Compilador



Análise Léxica

A primeira fase de um compilador é chamada de análise léxica ou leitura (scanning).

O analisador léxico lê o fluxo de caracteres que compõem o programa fonte e os agrupa em sequências significativas, chamadas lexemas.

Para cada lexema, o analisador léxico produz como saída um token no formato:

<nome-token, valor-atributo>

Que ele passa para a fase subsequente, a análise sintática. Em um token, o primeiro componente, **nome-token**, é um símbolo abstrato que é usado durante a análise sintática, e o segundo componente, **valor-atributo**, aponta para uma entrada na tabela de símbolos referente a esse token.

Análise Léxica (Exemplo)

`position = initial + rate * 60`

- ❑ “position” é um lexema mapeado em um token `<id, 1>`, onde id é um símbolo abstrato que significa identificador e 1 aponta para a entrada da tabela de símbolos onde se encontra position.
- ❑ O símbolo de atribuição “=” é um lexema mapeado para o token `<=>`. Como esse token não precisa de um valor de atributo, omite-se o segundo componente.
- ❑ “initial” é um lexema mapeado para o token `<id, 2>`, onde 2 aponta para a entrada da tabela de símbolos onde se encontra initial.
- ❑ “+” é um lexema mapeado para o token `<+>`.
- ❑ “rate” é um lexema mapeado para o token `<id, 3>`, onde o valor 3 aponta para a entrada da tabela de símbolos onde se encontra rate.
- ❑ “*” é um lexema mapeado para o token `<*>`.
- ❑ “60” é um lexema mapeado para o token `<60>`.

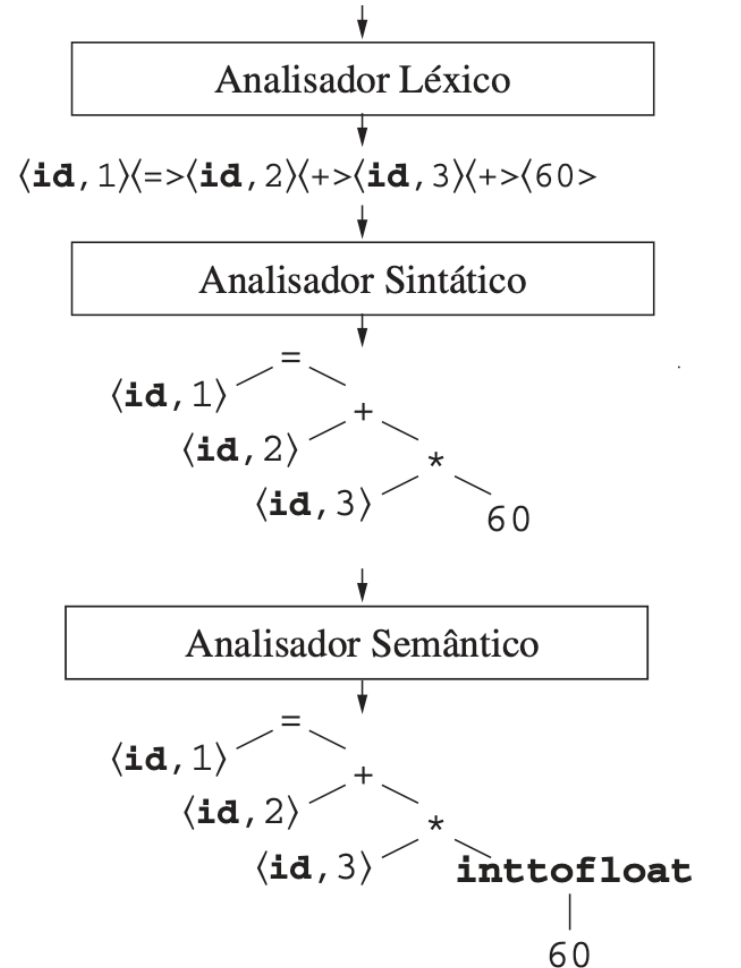
`•<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>`

Análise Sintática (Parsing)

O analisador sintático utiliza os primeiros componentes dos tokens produzidos pelo analisador léxico para criar uma representação intermediária tipo árvore, que mostra a estrutura gramatical da sequência de tokens.

Uma representação típica é uma **árvore de sintaxe (ou derivação)** em que cada nó interior representa uma operação, e os filhos do nó representam os argumentos da operação.

position = initial + rate * 60



Análise Semântica

O analisador semântico utiliza a árvore de sintaxe e as informações na tabela de símbolos para verificar a **consistência semântica do programa fonte com a definição da linguagem**.

Ele também reúne informações sobre os tipos e as salva na árvore de sintaxe ou na tabela de símbolos, para uso subsequente durante a geração de código intermediário.

Uma parte importante da análise semântica é a verificação de tipo, em que o compilador verifica se cada operador possui operandos compatíveis.

```
position = initial + rate * 60
```

Analizador Léxico

```
<id,1><=><id,2><+><id,3><+><60>
```

Analizador Sintático

```
graph TD
    A["<id,1>"] --- B["="]
    C["<id,2>"] --- B
    B --- D["+"]
    D --- E["<id,3>"]
    D --- F["*"]
    F --- G["60"]
```

Analizador Semântico

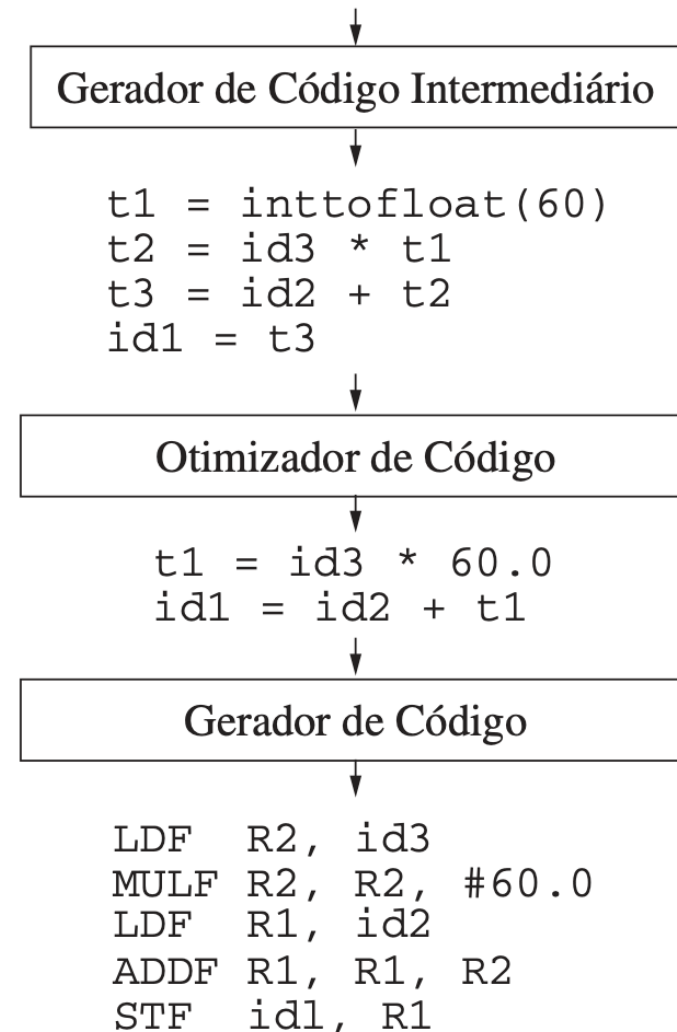
```
graph TD
    A["<id,1>"] --- B["="]
    C["<id,2>"] --- B
    B --- D["+"]
    D --- E["<id,3>"]
    D --- F["*"]
    F --- G["inttofloat"]
    G --- H["60"]
```

Geração de Código Intermediário

Depois das análises sintática e semântica do programa fonte, muitos compiladores geram uma representação intermediária explícita de baixo nível ou do tipo linguagem de máquina, que podemos imaginar como um programa para uma máquina abstrata.

Essa representação intermediária deve ter duas propriedades importantes: ser facilmente produzida e ser facilmente traduzida para a máquina alvo.

Ex: Código de 3 endereços.



Otimização de Código

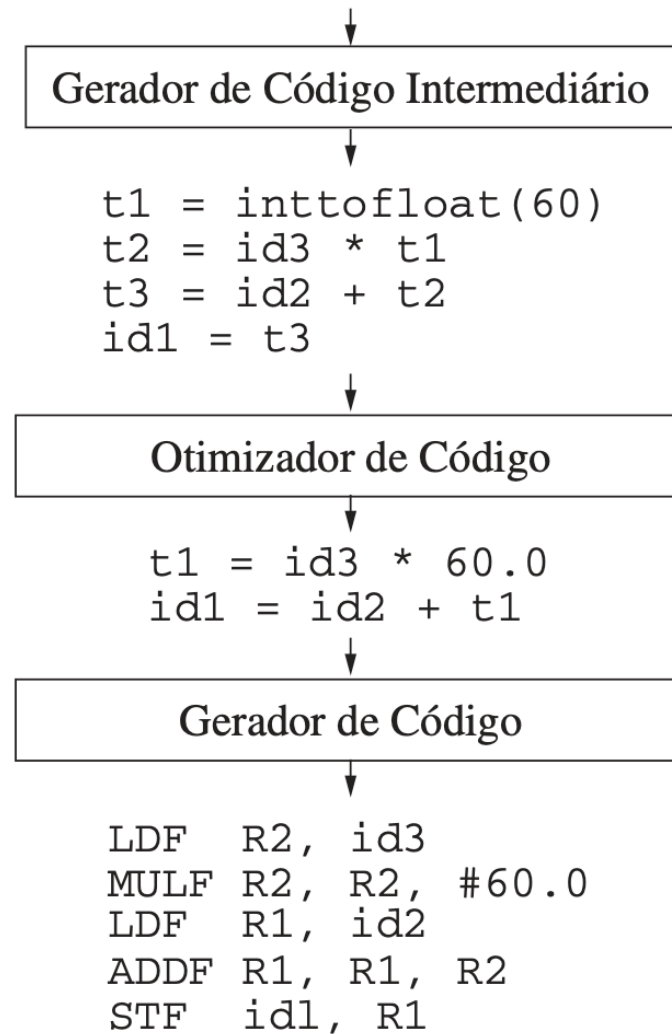
A fase de otimização de código independente das arquiteturas de máquina faz algumas transformações no código intermediário com o objetivo de produzir um código objeto melhor. Normalmente, melhor significa mais rápido, mas outros objetivos podem ser desejados, como um código menor ou um código objeto que consuma menos energia.

De:

```
t1 = inttfloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Para:

```
t1 = id3 * 60.0
id1 = id2 + t1
```



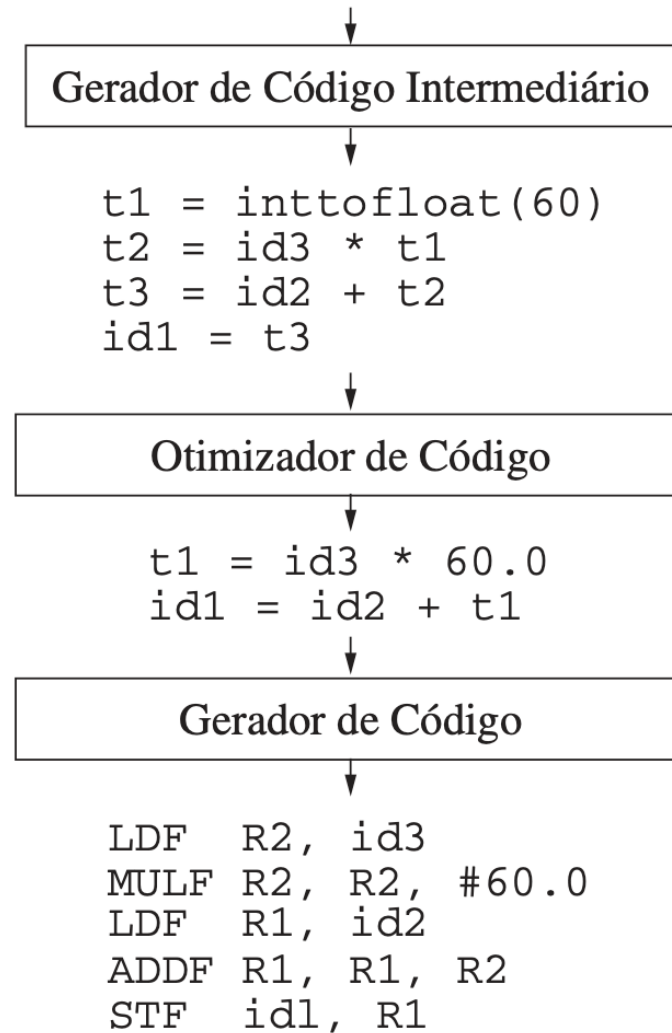
Geração de Código

O gerador de código recebe como entrada uma representação intermediária do programa fonte e o mapeia em uma linguagem objeto.

Se a linguagem objeto for código de máquina de alguma arquitetura, devem-se selecionar os registradores ou localizações de memória para cada uma das variáveis usadas pelo programa.

Depois, os códigos intermediários são traduzidos em sequências de instruções de máquina que realizam a mesma tarefa.

Um aspecto crítico da geração de código está relacionado à cuidadosa atribuição dos registradores às variáveis do programa.



Evolução das Linguagens de Programação

1ª Geração: Ling. de máquina.

2ª Geração (Baixo Nível): Assembler.

3ª Geração (Alto Nível): Fortran | Cobol | C | JAVA.

4ª Geração (Produtividade): SQL, MATLAB.

5ª Geração (Lógica): Prolog, OPS5.

Projeto de novas Arquiteturas de Computador

No desenvolvimento de arquiteturas de computadores modernas, os compiladores são desenvolvidos no estágio de projeto do processador, e o código compilado, executando em simuladores, é usado para avaliar os recursos arquitetônicos propostos.

RISC (Reduced Instruction-Set
Computer)

X

CISC (Complex Instruction-Set
Computer)

RISC -> PowerPC, SPARC,
MIPS, Alpha e PA-RISC

CISC -> x86 (Usa ideias do
RISC)

Síntese de Hardware ->
Verilog, VHDL

Interpretador

❑ **Interpretação:** é o processo em que um programa (o interpretador) lê, analisa e executa o código diretamente, instrução por instrução, **sem gerar previamente um arquivo executável independente.**

❑ **Exemplos:**

- ❑ Python
- ❑ JavaScript
- ❑ PHP
- ❑ Ruby
- ❑ Perl
- ❑ Lua
- ❑ R
- ❑ Bash
- ❑ PowerShell

Compilação vs. interpretação

	Compilação	Interpretação
Vantagens	✓ a execução do código convertido é mais rápida; ✓ apenas o usuário precisa ter o compilador - o usuário final pode usar o código sem ele; ✓ o código traduzido é armazenado usando linguagem de máquina - como é muito difícil de entender, suas próprias invenções e truques de programação provavelmente permanecerão em segredo.	✓ você pode executar o código assim que tiver concluído - não há fases adicionais de conversão; ✓ o código é armazenado usando linguagem de programação, e não linguagem de máquina - isso significa que pode ser executado em computadores que usam linguagens de máquina diferentes; você não compila seu código separadamente para cada arquitetura.
Desvantagens	✗ a compilação em si pode ser um processo muito demorado - talvez você não consiga executar o código imediatamente após fazer uma alteração; ✗ você precisa ter tantos compiladores quanto plataformas de hardware nas quais deseja que seu código seja executado.	✗ não espere que a interpretação aumente seu código a alta velocidade - seu código vai compartilhar a capacidade do computador com o intérprete, então não pode ser muito rápido; ✗ Você e o usuário final precisam ter o intérprete para executar o código.

Linguagem de Montagem (Assembly)

❑ Montador (Assembler): programa que traduz código escrito em linguagem Assembly para código de máquina específico de uma arquitetura de processador.

❑ Ex:

MOV AX, BX => 89 D8

ADD AX, 10 => 83 C0 0A

❑ Em geral, cada instrução da linguagem de montagem é traduzida em uma instrução de máquina pelo montador (1:1). É uma linguagem dependente do hardware, ou seja, há uma linguagem de montagem diferente para cada tipo de processador.

Linguagem de Montagem (Assembly)

Registradores de uso geral:

- ❑ **EAX**: acumulador, usado em operações aritméticas;
- ❑ **ECX**: contador, usado em loops;
- ❑ **EDX**: registrador de dados, usado em operações de entrada/saída e em multiplicações e divisões. É também uma extensão do acumulador;
- ❑ **EBX**: base, usado para apontar para dados no segmento DS (data segment);
- ❑ **ESP**: apontador da pilha (Stack Pointer), aponta para o topo da pilha (endereço mais baixo dos elementos da pilha);
- ❑ **EBP**: apontador da base do frame, usado para acessar argumentos de procedimentos passados pela pilha;
- ❑ **ESI**: índice da fonte de dados a copiar (Source Index), aponta para dados a copiar para DS:EDI;
- ❑ **EDI**: índice do destino de dados a copiar (Destination Index), aponta para o destino dos dados a copiar de DS:ESI.

Linguagem de Máquina

- ❑ Cada instrução de máquina deve conter as informações exigidas pelo processador para a execução.
- ❑ Instrução de máquina:
 - ❑ Código de operação (opcode).
 - ❑ Referência a operando fonte (registradores, endereço de memória, imediato, dispositivo de E/S).
 - ❑ Referência a operando de resultado.
 - ❑ Referência à próxima instrução.

JAVA

Compilação -> gera Bytecode -> Interpretado pela JVM.

```
public class OlaMundo {  
    public static void main(String[] args) {  
        System.out.println("Olá, mundo!");  
    }  
}
```

(Olamundo.java)

Javac

OlaMundo.class
(bytecode)

JVM

JVM

|

v

Código de Máquina

|

v

Processador

JAVA

Versão	Lançamento	LTS?	Situação
Java 8	2014	✓	Muito utilizado em sistemas legados
Java 11	2018	✓	Muito utilizado em sistemas corporativos
Java 17	2021	✓	Muito utilizado atualmente
Java 21	2023	✓	LTS anterior
Java 25	2025	✓	LTS atual
Java 26	2026	✗	Versão mais recente
Java 27	2026	✗	Previsto para setembro/2026

LTE = Long-Term Support

Atividade Avaliativa 1

Utilizando o Assembler NASM (Windows)



PRINCIPAIS ASSEMBLERS

ARQUITETURAS, FERRAMENTAS E CARACTERÍSTICAS

Assemblers (ou montadores) traduzem código Assembly para linguagem de máquina específica de uma arquitetura de processador. Eles são componentes essenciais no desenvolvimento de software de baixo nível, sistemas embarcados, SOs, compiladores e aplicações de alto desempenho.

CARACTERÍSTICAS COMUNS DOS ASSEMBLERS

-  Traduzem instruções Assembly em código de máquina.
-  São específicos para cada arquitetura de processador.
-  Podem gerar arquivos objeto (.o), executáveis ou código binário.
-  Suportam diretivas (seções, dados, definições) e macros.
-  Integram-se a linkadores e depuradores.

ARQUITETURA	PRINCIPAIS ASSEMBLERS	DESENVOLVEDOR / ORIGEM	SISTEMAS OPERACIONAIS	SINTAXE PADRÃO*	GERAÇÃO DE SAÍDA	USOS MAIS COMUNS
 x86 / x86-64 (32 e 64 bits)	• NASM (Netwide Assembler) • MASM (Microsoft Macro Assembler) • FASM (Flat Assembler) • YASM (Yet Another Assembler) • GAS (GNU Assembler - as) • TASM (Turbo Assembler)	• Netwide / Comunidade • Microsoft • Tomasz Grysztar • Projeto Yasm • FSF (GNU Binutils) • Borland (1992)	• Windows, Linux, macOS • Windows (Visual Studio) • Windows, Linux • Multiplataforma • Linux, *BSD, macOS, Windows (MinGW) • DOS, Windows (antigo)	• Intel • Intel • Intel • Intel • AT&T (padrão), Intel (opcional) • Intel	• .o, .obj, .bin, .exe • .obj, .lib, .exe • .exe, .bin • .o, .obj, .bin • .o, .obj, .so, .elf • .obj, .exe, .com	• Sistemas, drivers, games, perf, crítica, ensino • Desenvolvimento Windows, aplicações nativas • Executáveis pequenos e rápidos • Projetos multimídia (FFmpeg, etc.) • Ambientes GNU/Linux e multiplataforma • Desenvolvimento legado
 ARM (32 e 64 bits)	• ARMASM / ARM Compiler armasm • GAS (GNU Assembler) • LLVM Integrated Assembler	• Arm (antigo ARM Ltd.) • FSF (GNU Binutils) • LLVM Project	• Windows, Linux • Linux, *BSD, macOS, Windows (MinGW) • Multiplataforma	• ARM • AT&T (padrão), ARM (opcional) • ARM	• .o, .axf, .elf • .o, .elf, .so • .o, .obj	• Sistemas embarcados, dispositivos móveis • Ambientes GNU/Linux • Compiladores LLVM/Clang, tools modernos
 MIPS (32 e 64 bits)	• MARS (educacional) • SPIM (educacional) • GAS (GNU Assembler)	• Universidade de Melbourne • Universidade de Rice • FSF (GNU Binutils)	• Windows, macOS, Linux • Multiplataforma • Linux, *BSD, macOS, Windows (MinGW)	• MIPS • MIPS • AT&T (padrão)	• (simulador) • (simulador) • .o, .elf, .so	• Ensino e aprendizado de Arquitetura • Ensino e aprendizado de Arquitetura • Desenvolvimento real em plataformas MIPS
 RISC-V (32, 64 e 128 bits)	• RARS (educacional) • GAS (GNU Assembler) • LLVM Integrated Assembler	• Universidade de Auckland • FSF (GNU Binutils) • LLVM Project	• Windows, macOS, Linux • Linux, *BSD, macOS, Windows (MinGW) • Multiplataforma	• RISC-V • AT&T (padrão), RISC-V (opcional) • RISC-V	• (simulador) • .o, .elf, .so • .o, .obj	• Ensino e aprendizado de Arquitetura • Desenvolvimento real RISC-V (Linux, IoT, SoCs) • Ferramentas modernas e multiplataforma
 PowerPC / PPC (32 e 64 bits)	• GAS (GNU Assembler) • LLVM Integrated Assembler	• FSF (GNU Binutils) • LLVM Project	• Linux, *BSD, macOS, Windows (MinGW) • Multiplataforma	• AT&T (padrão), PowerPC (opcional) • PowerPC	• .o, .elf • .o, .obj	• Sistemas embarcados, consoles, servidores • Ambientes modernos baseados em LLVM
 AVR (8 bits)	• AVR Assembler (avrasms2) • GAS (GNU Assembler)	• Microchip / Atmel • FSF (GNU Binutils)	• Windows, Linux, macOS • Linux, *BSD, macOS, Windows (MinGW)	• AVR • AVR (via GAS)	• .o, .elf, .hex • .o, .elf	• Desenvolvimento para Arduino e sistemas embarcados • Ambientes GNU/Linux
 Outras arquiteturas (exemplos populares)	• SPARC: GAS, Oracle Assembler • IBM z/Architecture: HLASM • 8051: AS31, AS1 • PIC (Microchip): MPASM, XC8 Assembler	• SPARC / Oracle • IBM • Keil, Atmel • Microchip	• SPARC / Oracle • IBM • Keil, Atmel • Microchip	• Varia (SPARC, HLASM, Intel, etc.)	• .o, .elf, .bin, .list, .hex, etc.	• Mainframes, sistemas legados, IoT, automação, controle industrial e muito mais

Assembler NASM

O **NASM (Netwide Assembler)** é um dos assemblers mais populares para a arquitetura **x86 e x86-64**. É gratuito, de código aberto e amplamente utilizado em cursos de Arquitetura de Computadores, Sistemas Operacionais e Programação em Assembly.

Download:

<https://www.nasm.us/pub/nasm/releasebuilds/3.02rc13/>

Arquivo ZIP

Extraia o conteúdo para uma pasta, por exemplo:

C:\nasm

Adicione essa pasta ao **PATH** do Windows:

Configurações → Sistema → Sobre → Configurações Avançadas do Sistema

Clique em **Variáveis de Ambiente**

Edite a variável **Path**

Adicione:

C:\nasm

Windows / MSYS2 (GCC)

Acessar o site e baixar o instalador para Windows:

https://www.mingw-w64.org/getting-started/msys2/?utm_source=chatgpt.com

Instale normalmente, abra o terminal do **MSYS2 UCRT64** e instale:

```
pacman -Syu
```

```
// Aceite as sugestões, pode ser necessário reiniciar
```

```
pacman -S mingw-w64-ucrt-x86_64-gcc
```

```
gcc --version
```

```
// gcc.exe (Rev5, Built by MSYS2 project) 16.1.0
```

Esse método instala:

☐ GCC (C)

☐ G++ (C++)

☐ GDB (depurador)

☐ Make

☐ Binutils

Hello.asm

```
; hello.asm
; Exemplo simples para Windows (x64)
default rel
extern ExitProcess
extern printf
section .data
    mensagem db "Ola, mundo!", 10, 0
section .text
global main
main:
    sub rsp, 40      ; Alinhamento da pilha (ABI do Windows x64)
    lea rcx, [mensagem]
    call printf
    xor ecx, ecx
    call ExitProcess
```


Arquivos Objetos e Binários

Execute:

```
nasm -f win64 hello.asm -o hello.obj  
// O arquivo objeto será gerado  
gcc hello.obj -o hello.exe  
./hello.exe
```

Atividade Avaliativa

1. Documente todos os passos realizados.
2. O que é o arquivo Objeto gerado?
3. O que é o arquivo Binário gerado?
4. O que o programa faz? Explique as instruções utilizadas.

Envie um relatório das atividades realizadas para o NEAD.